Bachelor Thesis

Autonomous Steering of a Magnetic Catheter for Spinal Cord Stimulation Procedure

Elias Steiner

Harun Torlakcik

Prof. Dr. Salvador Pané i Vidal

Adviser

Prof. Dr. Bradley J. Nelson

Institute of Robotics and Intelligent Systems

Swiss Federal Institute of Technology Zurich (ETH)

2022-01

Institute of Robotics and Intelligent Systems

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Preface

I would like to thank Prof. Dr. Bradley J. Nelson, Prof. Dr. Salvador Pané i Vidal and my supervisor Harun Torlakcik for the opportunity of working on this project. I thank you for providing me with your knowledge and expertise throughout working this thesis. Additionally, special thanks go to Dr. Quentin Boehler for his patient support with the software implementation. Further, I would like to thank Sean Lyttle and Erdem Can Siringil for helping me on numerous occasions and providing their technical support.

Abstract

Chronic pain is an emergent problem affecting a substantial amount of people across the globe. Among all types of chronic pain spinal pain is one of the most common. Often it is refractory to medical treatment and even re-operation is seldom successful. Spinal Cord Stimulation (SCS) provides a viable solution in that case and can alleviate chronic pain even when other treatments fail.

While SCS is becoming more reputable it is not yet without flaws: especially during the implantation surgery complications can occur. The catheter implant is hard to manually control since it is inside of the human body and therefore physical trauma is not uncommon due to the catheter accidentally puncturing tissue. Additionally, surgeons receive incredibly high doses of radiation while performing the necessary fluoroscopy for this procedure. Therefore the aim of this project is to improve the SCS-procedure: a magnetically steered catheter combined with advancer-units enables the remote execution of this procedure and allows for precise control over the catheter, even once it is inside the epidural space. Finally a control algorithm will automatically manage the catheter steering and make the procedure simpler and more reliable. Through an already existing fluoroscope the controller can obtain visual feedback over the catheters position and operates in a closed-loop fashion.

With the help of specifically built silicone phantoms of the epidural space a prototype setup has been tested and further development possibilities have been layed out.

Contents

List of Tables **viii**

List of Figures **x**

Notation **xi**

1 Introduction **1**

2 Modelling the Epidural Space **7**
2.1 Theory 7
2.2 Abstract Model 7
2.2.1 Requirements and Design 7
2.2.2 Materials and Methods 9
2.3 3D-Model 11
2.3.1 Requirements and Design 11
2.3.2 Materials and Methods 12
2.4 Replication of Epidural Adipose Tissue 13
2.5 Results and Discussion 14

3 Modifying the Catheter **17**
3.1 Theory 17
3.2 Requirements and Design 17
3.3 Materials and Methods 18
3.4 Results and Discussion 19

4 Advancer Units **23**
4.1 Theory 23
4.2 Requirements and Design 25
4.3 Materials and Methods 26
4.4 Results and Discussion 27

5 Software Implementation **28**
5.1 Theory 28
5.2 Tracking Node 30
5.3 Control Node 36

5.4 Results and Discussion 38

6 Experiments **41**
6.1 Full Controller Test in Abstract Phantom 41
6.1.1 Motivation 41
6.1.2 Experimental Setup 41
6.1.3 Results and Discussion 42

7 Conclusion and Outlook **44**
7.1 Conclusion 44
7.2 Outlook 45

References **50**

A Appendix **51**
A.1 Tracking-Node source-code 51
A.2 Control-Node source-code 64
A.3 X-ray Emulator-Node source-code 70

List of Tables

List of Figures

1 An SCS-implant . . . 3
2 SCS electrode positioning . . . 3
3 System Overview . . . 5
4 Abstract phantom CAD model . . . 8
5 Abstract phantom 3D printed . . . 9
6 Abstract phantom finished . . . 10
7 3D phantom CAD model . . . 12
8 3D Model test print . . . 13
9 Abstract phantom: different Versions . . . 15
10 Assembling the magnetic tip . . . 18
11 Brass mold for tip silicone coating . . . 19
12 Silicone coating on tip-extension . . . 19
13 Magnetic tips, all variants . . . 20
14 Magnetic tip bending tests: different magnet layouts . . . 20
15 Magnetic tip bending tests: different wire diameters . . . 21
16 Magnetic tip model test, setup . . . 22
17 Magnetic tip model test, sharp bend . . . 22
18 Magnetic tip final version . . . 22
19 Existing advancer-unit . . . 23
20 Advancer layout . . . 24
21 Advancer sheath guides: for catheter . . . 25
22 Advancer sheath guides: for guidewire . . . 25
23 Finished advancer modifications . . . 26
24 ROS philosophy . . . 28
25 ROS implementation structure . . . 29
26 Blob-tracker extracted info overview . . . 30
27 Blob-tracking parameters . . . 31
28 Model markers . . . 32
29 Naion workspace markers . . . 33
30 Catheter characteristics . . . 34
31 Catheter initial chains . . . 34
32 Catheter expansion algorithm . . . 35
33 Blob-tracker overlay . . . 36

34 Catheter stuck in model 37
35 The experimental setup 42
36 Experiment user interface 43

Notation

The following abbreviations will be used throughout this thesis:

CA Cyanoacrylate.

CAD Computer Aided Design.

FBSS Failed Back Surgery Syndrome.

FDM Fused Deposition Modelling.

fps Frames per Second.

ID Inner Diameter.

MMU Multi-Material-Upgrade.

MRI Magnetic Resonance Imaging.

MSRL Multi Scale Robotics Lab.

Nitinol Nickel-Titanium Alloy.

OD Outer Diameter.

PLA Polylactic Acid.

PVA Polyvinyl Acetate.

PVB Polyvinylbutyral.

ROS Robot Operating System.

RViz ROS Visualization.

SCS Spinal Cord Stimulation.

SLS Selective Laser Sintering.

STL Standard Tesselation Language.

TPU Thermoplastic Polyurethane.

1 Introduction

"Low back pain is a worldwide problem, with an estimated 9.4% global incidence, creating more disability than any other condition in the world" [2]

Similar statistical findings are acknowledged by a variety of studies around the globe [18]: for example, in the United States around 20% of all adults suffer from general chronic pain and 8% even experience high-intensity chronic pain [9]. These high percentages clearly demonstrate, how prominent of an issue chronic pain is in present-day population.

Root Causes and Problems Chronic pain commonly emerges after accidents, diseases or failed surgical procedures. Initially, pain can be directly caused by damaged tissue and serves as a biological signal to indicate the damage. Once the healing process has concluded, such pain is expected to recede. However, if pain prevails long after any tissue damage has healed, it is called chronic pain and is therefore seen as "useless" and serves no biological purpose. Since treatable causes are missing, symptomatic therapy is therefore administered to improve the patients situation [10].

Among all types of chronic pain, spinal pain represents a significant proportion of cases. While there are other root causes, Failed Back Surgery Syndrome (FBSS) is by far the most common source for chronic back pain [15]. FBSS constitutes all cases, where patients have undergone surgery to alleviate back pain and experienced a negative outcome: either they see no improvement at all or return to a state of chronic pain in the same region around 3-6 months after surgery. Around 30% of patients undergoing any kind of back surgery experience this syndrome. [2]. Unfortunately, chronic pain is often refractory to medical treatment and re-operation seldom succeeds. Patients find themselves with very limited options to mitigate their suffering. In that case, Spinal Cord Stimulation (SCS) can provide an effective solution to sustainably improve the situation of chronic patients [26].

SCS-Procedure and its Effects SCS is a minimally invasive procedure and the most implemented practice of Neuromodulation. These are medical interventions involving the manipulation of nerve impulses by either electrical or medical means [36, 21]. SCS requires the implantation of electrodes in close

proximity to the spinal cord. By applying electrical signals to these electrodes, nerve transmissions passing through the spinal cord can be disturbed, including chronic pain for example. The interfering signals are generated by a unit similar to a pacemaker, which is connected to the electrodes via long leads. Signal characteristics can vary depending on the treatment method applied. Nowadays, mainly two variants of SCS-procedures exist [29, 12, 21]:

- The original low-frequency Spinal Cord Stimulation, where electric impulse signals range from 40-70 Hz.
- The newer high-frequency Spinal Cord Stimulation, which works with signals around 10 kHz.

Patients who receive low-frequency SCS treatment experience a feeling which is described as a "tingling sensation" once the electrodes are activated. This so called paresthesia replaces any sensation of chronic pain and provides sustained relief for patients [29, 15, 21]. Studies find that around 47% of patients, who undergo SCS treatment experience at least 50% sustained pain relief. This amount of improvement is considered a sucessful application. In contrast, re-operation manages to achieve a satisfying outcome in only about 12% of all cases [26, 21].

Lately, high-frequency SCS has become more and more popular, since it obtains similar pain-suppressing results without evoking paresthesia. While this is already an improvement, ongoing research is experimenting with different types of disturbance signals to improve SCS even further [29, 15, 26]. In this thesis, work will be focused only on high-frequency SCS, because each approach requires a particular type of implant placed in distinct locations.

High-Frequency SCS When a patient undergoes high-frequency SCS treatment, the implantation of two separate, elongated electrodes containing around 5-20 individual contacts is required. They are installed on flexible plastic catheters which also contain contact leads to establish a connection to the signal generator. The catheters are around 50-70 cm long and measure 1-2 mm in diameter. These electrodes are placed inside of the epidural space in close proximity to the spinal cord.

During surgery a hollow needle called Tuohy needle is first inserted into the spinal canal between two vertebrae of the lumbar region. The electrode-catheter

can be pushed through the needle and enter the epidural space. Manually, a surgeon will then advance this catheter along the spinal cord until the electrodes reach an area between the Th8 to Th12 vertebrae. The catheter can be controlled by pushing and pulling on the end, which protrudes from the body. At all times the surgeon will observe the location of the catheter-electrode through fluoroscopy, a sort of continuous x-ray [12, 33, 35, 21].



Figure 1: The SCS implant consisting of two electrode leads and a signal generator [7]

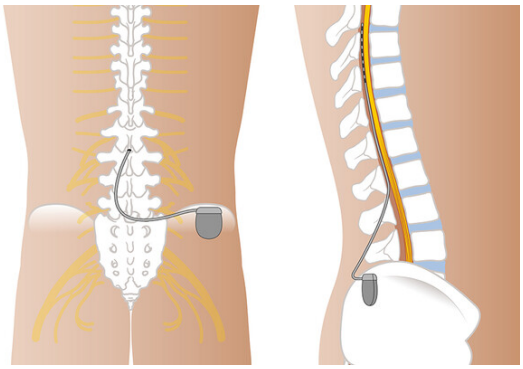


Figure 2: Image showing the electrode position inside of the epidural space [21]

After both electrodes have been implanted, a signal generator is temporarily connected to the protruding ends of the catheters. Patients will undergo a trial phase, where the optimal electrode placement and signal type is tested. After this phase is completed a permanent, battery powered version of the signal generator is also implanted. It is often situated near the hip or back region. This concludes the surgical part of an SCS-procedure.

Complications of SCS While the results found by clinical trials sound promising, SCS still has a few major flaws. Serious complications are rare and estimated to occur in fewer than 1% of all patients. However, less severe, but still displeasing problems develop in about 35% of all cases. Roughly 20% of those require surgical revision and therefore deteriorate the quality of life for patients. Common complications include [10, 21]:

- Trauma from electrode implantation. Especially puncturing blood vessels can lead to the formation of a hematoma, which requires additional medical attention.

- Various types of infections can lead to inflammation around the implant. This can occur either shortly after surgery and close to the catheter or long after the medical intervention and near the signal generator implantation site.
- Dural Puncture: damaging the protective tissue around the spinal cord.
- Epidural Fibrosis: the excessive production of scar tissue.
- Electrode Migration: catheters can move around due to it and the body being inherently flexible.

Some of those complications are solely preventable by either scrutinous examination for preexisting conditions or perfectly sterile execution of the implantation procedure to prevent infections. Other issues, for example electrode migration and traumatic damage to tissue during the implantation, are of more technical nature: namely the surgeon having very limited physical control over the catheter during or even after the surgery.

Magnetic Catheter Navigation: State of the Art Catheters are generally a widely used instrument in medicine. Altough, in their most basic form they are usually just a plastic or metal tube. When equipped with guide-wires and magnets they can become a powerful, modern surgical instrument. By using externally generated magnetic fields such a catheter can be steered remotely and with outstanding precision. This allows for otherwise complex operations to be carried out in a minimally invasive fashion. For example, surgical procedures inside of the heart can almost automatically thanks to magnetically guided catheters and control algorithms [14, 20].

Improving the SCS-Procedure Currently, electrode leads are controlled manually by their protruding ends during the surgery. Longitudinal motion is achieved by simply pushing an pulling the leads. In order to realize curves, the catheter is equipped with a bent tip. Through rotating the catheter, the surgeon can choose to alter its trajectory, since the catheter will always move towards the side its tip is pointing to [35, 33]. Whereas this allows the surgeon to place the lead precisely enough in order for the procedure to succeed, he has no direct control over the tip itself.

To improve physical control over the catheter during implantation, its tip should be equipped with powerful neodymium magnets. In combination with an externally generated magnetic field, this modification should facilitate immediate control over the tip bending angle without altering the medical procedure itself and making it more invasive. Fields will be generated using the "Navion" system at ETH Zürich. It can precisely induce magnetic fields in any direction inside of a 20x20 cm workspace at a maximal magnitude of around 10-30 mT [4, 3].

Human errors could be further reduced by introducing an automated setup to fully control catheter movement. This is achieved by using mechanical advancer-units to also control longitudinal motion of the electrodes besides the tip angle. Additionally, a control algorithm should coordinate both, the magnetic field and the advancers. By providing the fluoroscopy image as visual feedback to the algorithm it can operate in a closed-loop mode. This will not only improve the precision of the whole implantation, but also alleviate another, previously unmentioned problem: surgeons receive incredibly high doses of radiation when performing the fluoroscopy required for an SCS implantation [37, 24]. If the procedure were remote-controlled, no surgeon has to be present near the harmful radiation source. Automating the surgical procedure for SCS aims to improve both the surgeons and patients situation.

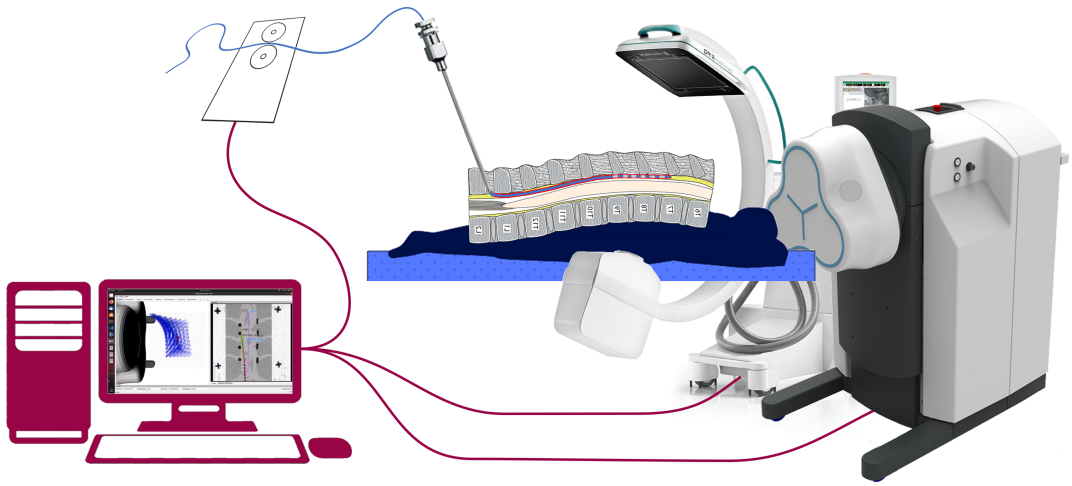


Figure 3: An overview over the new SCS-system

Project Outline Multiple individual elements have to be developed in order to realize all of the aforementioned improvements. Chapters two through five will each discuss one separate component:

In chapter 2 the design and manufacturing of silicone models - so called phantoms - of the epidural space will be discussed. They are essential for testing prototypes of the improved SCS-system. Chapter 3 examines the process of modifying an existing SCS implant and making it magnetically responsive. As described before, mechanical advancer-units will be needed to automatically move the catheter. An existing model from the Multi Scale Robotics Lab (MSRL) at ETH Zürich will be used. Small adjustments that had to be made to those units are documented in chapter 4. Lastly, chapter 5 illustrates how all components were brought together with the help of a control algorithm implemented in Robot Operating System (ROS). This includes visually tracking the catheter through a fluoroscope.

2 Modelling the Epidural Space

2.1 Theory

Most components of this thesis' work, like the modified implant or Python control-code, involved multiple iterations of theoretical ideas and physical prototypes. Akin to nearly all engineering projects, one thing would therefore become indispensable at some point: experimental testing. Without being able to experimentally validate or nullify ideas, progress is much harder to achieve. Testing, in this case, eventually involves inserting the modified catheter into a physical epidural space and examine its maneuverability. Unfortunately, human spines are not exactly readily available for scientific experiments, therefore, building a model replica, a phantom, of the epidural space was necessary.

Since organic shapes, like a human epidural space are inherently difficult to digitally model and tedious to manufacture, especially in full detail, it was beneficial to build two different kind of models: a more abstract, flat model for quick prototyping (refer to section 2.2 "Abstract Model"), as well as a more advanced, fully 3-dimensional phantom which captures intricate details of the epidural space (refer to section 2.3 "3D Model"). The latter should serve as a realistic testing scenario, once initial and easily determined flaws have been eliminated from the prototype system. The approach to designing and manufacturing such models will be discussed in this section.

2.2 Abstract Model

2.2.1 Requirements and Design

Outline As mentioned before, the abstract model should be relatively easy to manufacture, while still containing the most important features of the epidural space. For this reason, it was decided that 3D printing should be used as much as possible. Especially Fused Deposition Modelling (FDM) 3D printing is one of the quickest techniques to manufacture digitally designed objects containing reasonable levels of detail. With the help of this method, the desired part is built layer by layer from various types of extruded thermoplastics. The information about an object is transferred to the printer via so called Standard Tesselation Language (STL)-files which describine its surface in three dimensions.

Using FDM 3D printing a water-soluble negative of the epidural space will be fabricated. That model can then be cast in silicone rubber to imitate surrounding tissue. Once the silicone has cured the negative is dissolved, leaving behind a cavity in the shape of an abstract epidural space.

CAD Design As a first step a CAD model of the water-soluble part, containing the necessary details, had to be drawn. During an SCS procedure, the surgeon mainly steers the catheter towards the left or right side of the body apart from advancing it along the spinal cord [35, 33]. Therefore, even an abstract model should correctly depict the coronally projected shape of the epidural space. This is equivalent to viewing the patient directly from the back side. Dorsal root ganglions, which are the nerves branching off the spinal cord, should be included as well.

Any curvature out of this coronal plane can be omitted for now. Likewise, the depth of the epidural space is chosen to be a constant, average value of about 5 mm [22]. These simplifications lead to an essentially two-dimensional shape with constant depth, which is easily modelled and printed. Using anatomy references to achieve reasonable biological accordance [25, 19], a two-dimensional sketch of the epidural space was created in Siemens NX 12. Some artificial obstacles in the shape of ellipses and circles were added. Lastly, the sketch was then simply extruded to a depth of 5mm to obtain an abstract negative model of the epidural space, thus concluding the design.

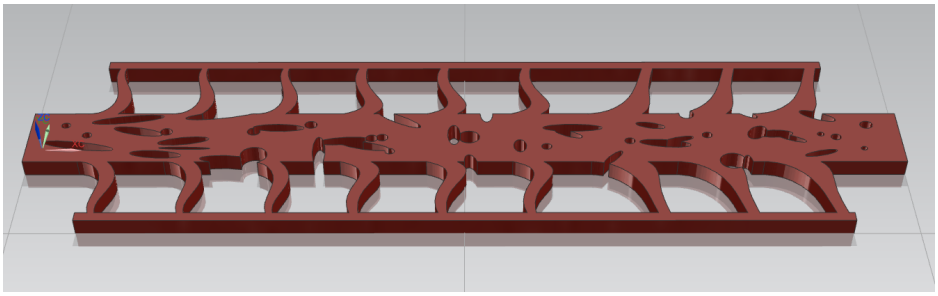


Figure 4: The rendered CAD model in Siemens NX 12

2.2.2 Materials and Methods

3D Printing The CAD model was then exported as an STL-file and printed on a Prusa i3 MK3S printer using Polyvinyl Acetate (PVA) filament. This type of thermoplastic was chosen due to its eligible water-soluble properties [17]. To improve the dissolution in water for subsequent steps, the part was printed mostly hollow (at 0% infill) with as little material as possible (1mm wall thickness).

3D printing tends to leave small imperfections on the finished model, such as strings or blobs of excess plastic. Due to that, the part had to be cleaned, first using a scalpel and tweezers to remove larger pieces. Sanding the part and quickly submerging it under a stream of warm water for about 2-3 seconds and then letting it dry yielded a smooth surface finish.

Encasing Using a laser cutter, walls for a cuboid case were cut from 3mm acrylic sheets. This box served to contain the liquid silicone and could be assembled around the 3D-printed part. Specially designed slots in the acrylic casing could accommodate parts of the plastic negative, thus holding it in place rigidly. To avoid spillage, the case was made watertight with the help of masking tape.

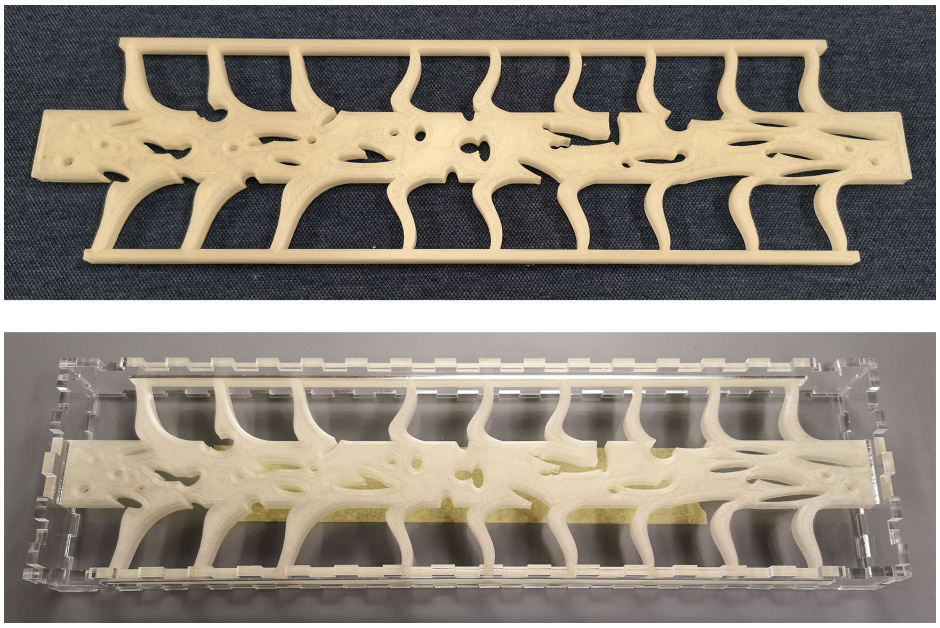


Figure 5: Shows the finished 3D printed model after cleanup and the acrylic box assembled around it

Casting the Silicone About 250ml of two-part silicone rubber (Altropol: Neukasil RTV 27 [1]) was then prepared and thoroughly degassed with the help of a vacuum chamber to avoid the formation of bubbles. After pouring this mixture into the acrylic box, containing the negative model, the silicone was left to cross-link for 24 hours. Conveniently, cured silicone will not stick to the acrylic sheet, which allowed for an easy disassembly of the box afterwards.

Dissolving PVA Finally, the silicone cast was submerged in warm water to dissolve all PVA structures, leaving behind only a cavity in the shape of an epidural space. This was done by simply immersing the phantom in a large beaker of warm water and replacing the water every 2 days. Unfortunately, the dissolution would often require more than a week. The cause for that was likely the low amount of PVA surface exposed to water due to the model being inside of the silicone and forming tight passages. Tests with bare PVA prints submerged in water revealed that under those circumstances, dissolution will only take about 12h.



Figure 6: Shows the finished abstract phantom after dissolution of any PVA

2.3 3D-Model

2.3.1 Requirements and Design

Outline While the abstract models allow for reasonable testing of the catheter maneuverability without investing excess amounts of time and resources, they still neglect more intricate and potentially important features: following the spine, the epidural space exhibits a subtle curvature towards the, previously neglected, third dimension. Additionally, its shape is generally more organic and has small indentations, where the spinal canal is pinched by the vertebrae. The influence of those features on the catheters movement should eventually be examined as well. This requires a precise, and fully detailed 3-dimensional model of the epidural space.

The approach to building such a phantom is analogous to the methods used for building the previously described abstract model: a water-soluble and 3D printed negative of the epidural cavity, will be cast in silicone rubber, with the only difference being the complexity and shape of the PVA negative. Additionally, the new model should also contain non-soluble plastic replicas of vertebrae, spinal discs, and the spinal cord to achieve a higher degree of realism.

CAD Design Again, an STL model was required for 3D printing. Unfortunately, no useful digital models containing all of the aforementioned components could be found. Therefore, it was necessary to create a custom CAD model of all the required parts. Luckily, an existing spinal model [30], consisting of vertebrae and intervertebral disks, could be found and was imported into Siemens NX 12 to serve as a foundation for further modelling.

To create the epidural space and spinal cord, regularly spaced coronal ("on a plane which is normal to the spinal canal") cross-sections of those features were drawn. These sketches could then be transformed into a volumetric body through the help of the "swept"-command. This command essentially interpolates all needed cross-sections to form a body from the given sections. Dorsal root ganglions were then added, in a similar fashion. Achieving reasonable biological accordance was possible, thanks to anatomy references, as well as other, existing 3D-models [25, 11, 19, 5, 28, 32, 23]. Of especially great help was one paper, which had managed to reproduce an accurate spinal cord-model through analyz-

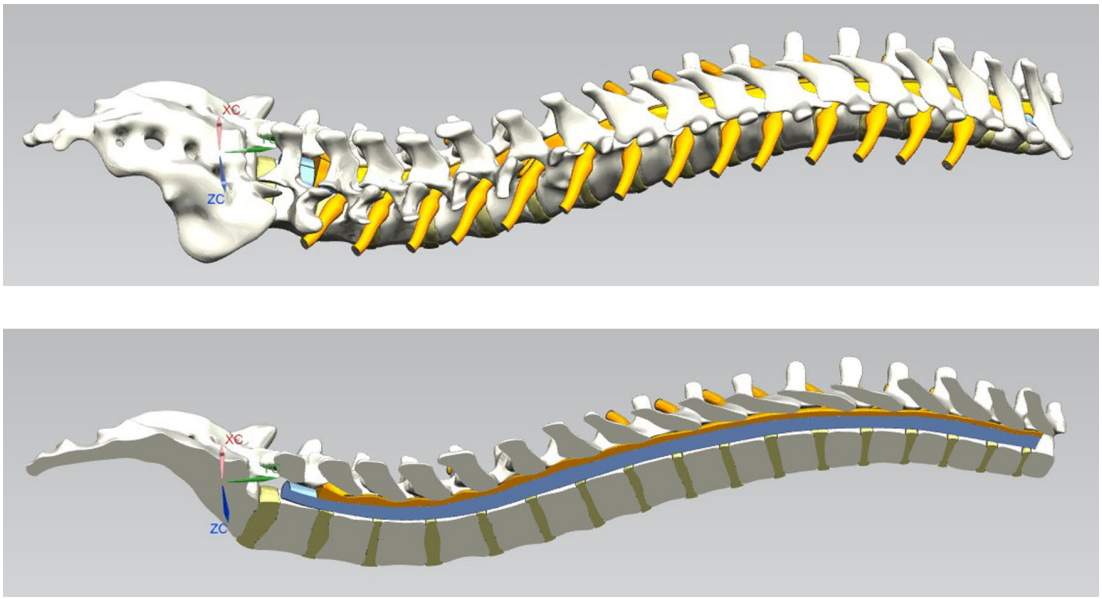


Figure 7: The finished custom CAD model for the 3D phantom

ing MRI-scans [32]. All CAD parts were then exported as individual STL-files with identical coordinate origins.

2.3.2 Materials and Methods

3D Printing Since this model contains the water-soluble epidural space, as well as permanent components like the vertebrae, spinal disks, and spinal cord, different materials had to be used as printing media. On a Prusa i3 MK3S, equipped with the Multi-Material-Upgrade (MMU), four different plastic filaments were prepared:

- Polylactic Acid (PLA) for printing hard and insoluble vertebrae bones.
- Thermoplastic Polyurethane (TPU) for printing flexible and insoluble spinal disks, as well as the spinal cord.
- PVA for printing the epidural space. This plastic is soluble in water but cannot be dissolved by acetone or isopropanol [17].
- Polyvinylbutyral (PVB) for printing support structures. Is soluble in many organic solvents, such as ethanol and isopropanol.

The whole model was then printed as one single piece containing the respectable materials mentioned above. Next, easily accessible parts of the PVB support structures were removed by hand and any hard to reach residues were dissolved with isopropanol. Due to time constraints, only a small test print could be fabricated. Printing the full structure and encasing it in silicone could not be completed.

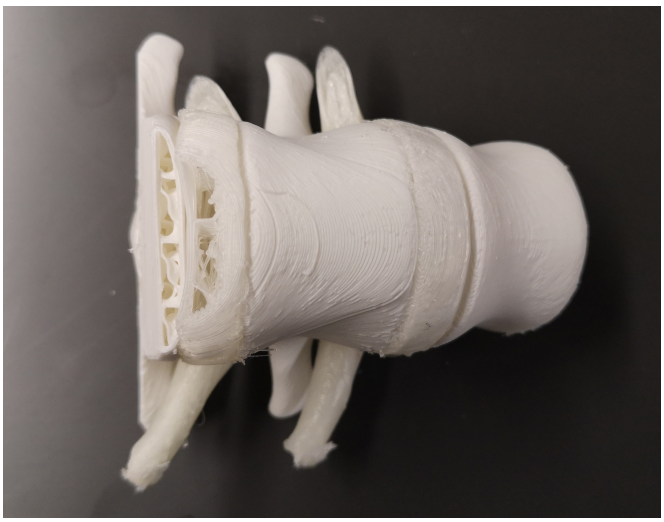


Figure 8: The test-print showing layer adhesion problems

2.4 Replication of Epidural Adipose Tissue

Motivation Until now, the epidural space has only been modelled as an empty cavity in silicone rubber. In reality however, this space is far from empty: it is filled with adipose tissue, blood vessels and varying amounts of connective tissue. Epidural adipose tissue, described as having a "yoghurt-like" consistency, only lightly influences the movement of a catheter. Blood vessels and especially connective tissue however, can be serious obstacles for the catheter and should therefore somehow be modelled as well.

Materials and Methods used Gelatin ("Gelatin from Porcine Skin, Sigma-Aldrich: 48724-500G-F) was used to mimic all three types of tissue as differently concentrated and solidified water-gelatin mixtures. Unfortunately, literature did not hold any exact concentration values for replicating the requested tissue properties with sucha gelatin-water mixture. To find a suitable consistency, five dif-

ferent batches of gelatin (1, 2, 3, 4, 5 wt%) were prepared by thoroughly mixing respective amounts of dry gelatin with pure 70°C water. After having solidified all variants were then tested, by manually shoving the catheter through the mixture and observing its behavior.

Gelatin at around 1 wt% was chosen to represent the adipose tissue. No trails were left by the catheter after moving through this material and it could still move through the gelatin in a straight path unguided. At around 3-5 wt% the catheter struggled to even puncture the material and could not at all be moved around inside. This property was used to represent blood vessels and connective tissue: small lumps (around 0.5 cm in diameter) of hardened 3, 4, and 5 wt% gelatin were added to the solidified 1 wt% mixture. This resulted in a mostly fluid substance which occasionally contains small obstacles.

Spoilage of Gelatin Being an organic substance, wetted gelatin will rapidly deteriorate and eventually grow mold after around one week. This behavior was repeatedly observed in all experiments involving gelatin. Vinegar was therefore added to the gelatin mixture as partial replacement for water at around 20 vol%. In total, 20% of the water was replaced by a solution of 5 vol% Acetic Acid (99.7%) in water. As suggested by the following paper [31], this should greatly improve the physical life span, by up to six weeks.

2.5 Results and Discussion

Abstract Phantom In total, three different versions of abstract models were built, each containing different amounts of obstacles. All of the other features are identical between the models. The two models containing the least amount of obstacles could both be finalized and were successfully used in numerous tests (figure 9, left and middle image). Almost no air inclusions were present in the silicone and all features were correctly molded as designed. Both of those models proved to be incredibly helpful while testing an numerous experiments were conducted with them. Without such models testing the catheter maneuverability or the final control algorithm would have been impossible.

The only fabrication step that proved to be a nuisance was the dissolution of PVA. As mentioned before, this step would often take more than one or even two weeks. Designing the PVA print to contain as little material as possible and

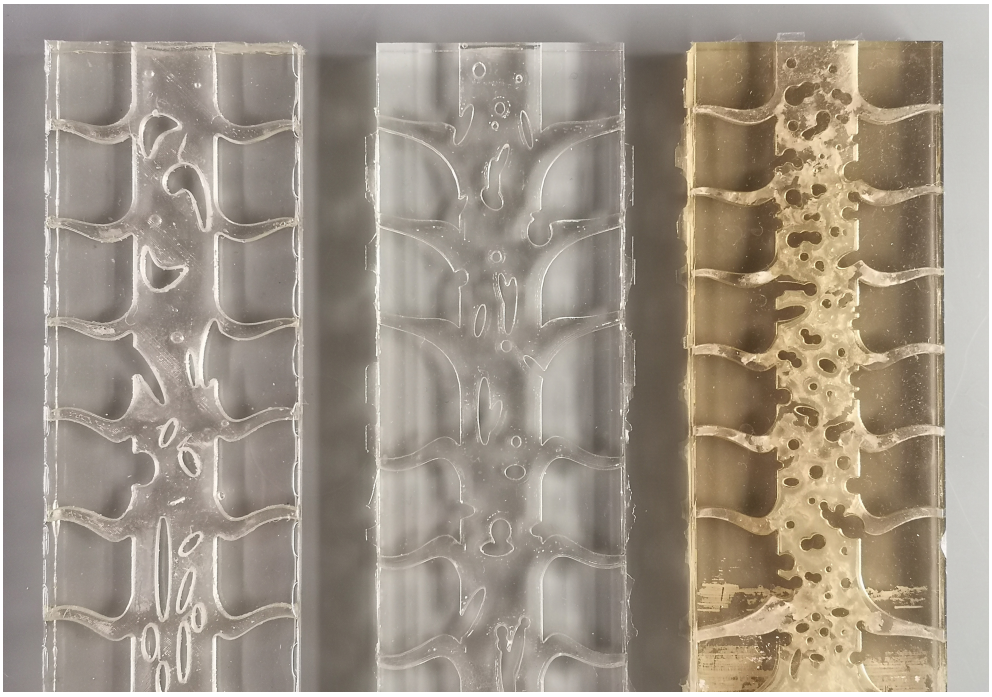


Figure 9: All three versions that were fabricated, containing various obstacle densities

making it hollow can speed up the dissolution rate significantly. Not doing so, in combination with having lots of obstacles inside of the epidural space phantom, will result in the plastic not dissolving at all. This is exactly what happened while manufacturing the rightmost version in figure 9.

3D Model Due to time constraints this model could not be completed. A few problems were encountered already while printing the negative 3D model: the layer adhesion was sometimes very poor and lead to splits in the finished parts. Carefully re-configuring the printers settings seemed to alleviate this issue. In addition to that, printing times are extremely long at up to a week for the entire model. This has to be taken into account carefully when planning to create such a model.

Replication of Epidural Tissue For the final control algorithm tests (refer to section 6.1) the abstract phantoms were used and filled with a 1.2 wt% gelatin mixture, which had 20 vol% of its water substituted by vinegar (5 vol% acetic acid) to improve longevity. By using this mixture to fill the otherwise

empty cavity, the catheter was slightly stabilized and could not move sideways as easily as it would without the gelatin. This served to make the experiment more realistic. However, the effect is not very pronounced. A denser gelatin mixture (1.5-2 wt%) is potentially better suited for these experiments since the vinegar slightly lessens the gelatins ability to coagulate. The physical longevity improvement of the gelatin due to the addition of vinegar could not be examined, since the improved shelf life can be as long as six weeks[31].

3 Modifying the Catheter

3.1 Theory

SCS-procedures themselves are a proven measure and thus have been in use for over 40 years now [29]. Consequently, numerous implant products have already been developed and were introduce to the marked by different companies. None of them, however, contain any magnets, that could be used to steer the catheter by using magnetic fields. Modifying an existing product was therefore necessary.

The model "Avista™ MRI, 74 cm" manufactured by Boston Scientific [6] was used for this project. At an overall length of 740 mm and a diameter of 1.6 mm it features a hollow and flexible catheter with eight electrodes at one end. During surgery the side containing the electrodes will be implanted, while the other end is left protruding and bears the contacts that can be connected to a signal generator. A Nitinol guidewire with a diameter of 0.36 mm runs through the whole catheter tube, providing stiffness. This guidewire can be retraced and inserted from the protruding end.

3.2 Requirements and Design

Requirements Most importantly, the tip (where the electrodes are located) of the previously described catheter had to be made magnetically responsive. When exposed to a magnetic field, generated by the Navion-system, it should ideally be able to bend with a curvature radius of about 15 mm. This is challenging, since the magnetic field cannot exceed 30 mT for a long duration due to current limitations in the coils used by the Navion [4, 3]. In addition to those requirements, any alterations to the existing implant cannot increase its outer diameter (1.5 mm) to more than 1.7 mm. This, in turn, is a consequence of the catheter having to fit through the Tuohy needle during surgery, when it is inserted into the epidural space. Similarly, any extensions should be as short as possible lengthwise since an excessive additional length could be obstructive during the implantation.

Design Taking these physical limitations into consideration, led to the conclusion that any extension would simply not contain enough volume to be very strongly attracted to the limited field of 30 mT. To still maximize the attractive

forces neodymium magnets were chosen for this modification. They provide an extremely high magnetic field strength per volume. Unfortunately, the catheter offered no redundant room to attach magnets directly, so an extension had to be built. A small hole to accommodate a wire was melted into the tip of the catheter using a hot needle. The magnets are then threaded onto a thin Nitinol wire and everything can be attached to the tip of the catheter using Cyanoacrylate (CA) glue.

3.3 Materials and Methods

Assembly Following those established design outlines a few slightly different magnetic catheter-extensions were built. While containing a varying number of magnets and differently sized wires, they were all assembled much in the same way:

First, about 100 mm of Nitinol wire (ordered from Chamfr [8]) were cut off and the ends ground to be smooth. A piece of Kapton tape was fixed to a work surface with the adhesive side facing upwards. Four to ten tubular Neodymium magnets (1.5mm x 1.4mm OD x 0.3mm ID) were then threaded onto the wire with all their polarities facing in the same direction. By pressing the magnets onto the Kapton tape and using metal rods as spacers, gaps between the magnets were introduced. Once an extension had been prepared with the correct number of magnets and spacing, they were individually glued to the wire permanently with CA glue.

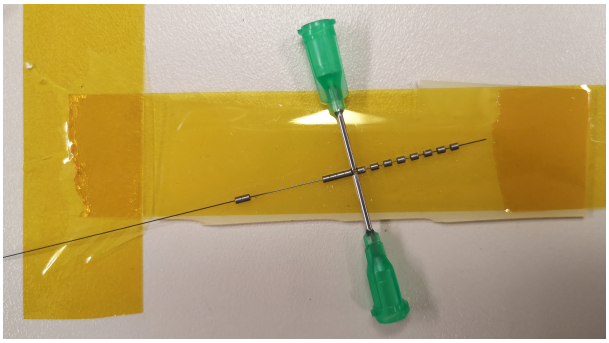


Figure 10: Holding the magnets in place with Kapton tape and spacers

Silicone Coating Since these magnetic tips are eventually attached to the leading end of the catheter, they will be in direct contact with the epidural tissue. To help the implant glide through tissue more smoothly and encase the non-bio-compatible magnets in a protective layer, they had to be coated with silicone rubber. This coating was required to be precise and uniform. Therefore a two-part mold was used to accurately manufacture such a coating.

In order to achieve the high precision and good surface finish needed for such a small cast, the mold was manufactured on a milling machine. If features two cuboid brass pieces which can be repeatedly aligned with respect to each other through the help of three steel registration pins. Both halves were pressed together and a hole with a diameter of 1.6mm and a depth of 30mm was drilled exactly through the separation plane of these two cuboids. This formed a cavity for the silicone and magnets that can be opened easily.

To apply the coating, the open mold as well as the previously assembled tip-extension had to be thoroughly covered in silicone before pressing together both halves of the mold around the magnets. For this, two-part silicone rubber (Dragon Skin 10A) was mixed and degassed thoroughly. After the silicone had cured, the mold was opened to release the cast and any excess silicone was removed carefully with a scalpel.

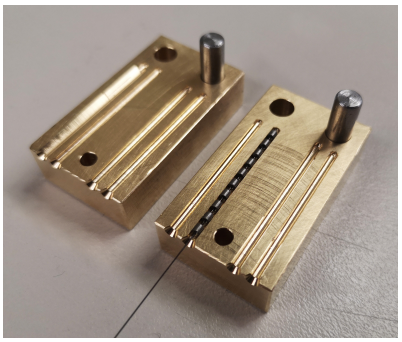


Figure 11: The opened brass mold



Figure 12: The tip-extension mounted on the catheter with silicone coating

3.4 Results and Discussion

Bending Radius Test: Magnet Layout As mentioned before, multiple variants of the tip extension were initially created. In all of them, either the

number of magnets or the spacing between magnets was altered. This was done to explore how different magnetic layouts would behave in the same magnetic field. For this first test, the wire diameter was kept constant in all versions. The following configurations were tested:

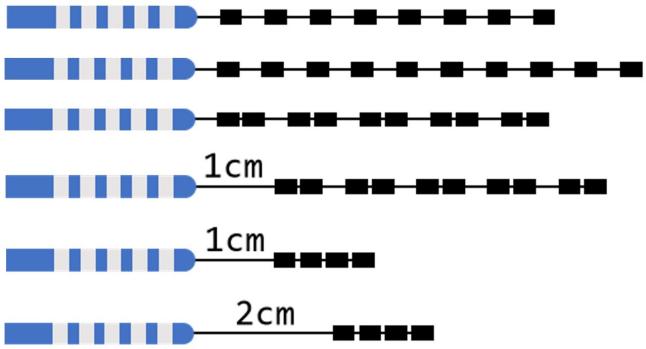


Figure 13: All the different types of magnetic tips that have been tested

Each tip was individually mounted on the catheter and clamped in a fixture. The setup was then exposed to magnetic fields in the Navion-workspace of strengths 10 mT, 20 mT, and 30 mT. During all tests, the field was pointing at a right angle to the catheter axis. An image was taken, for every individual setting from which later all the bending-angles could be determined with an angle gauge. From these results it was concluded that 10x1 magnets, mounted in equal succession, yields the tightest bending curvature for a given magnetic field.

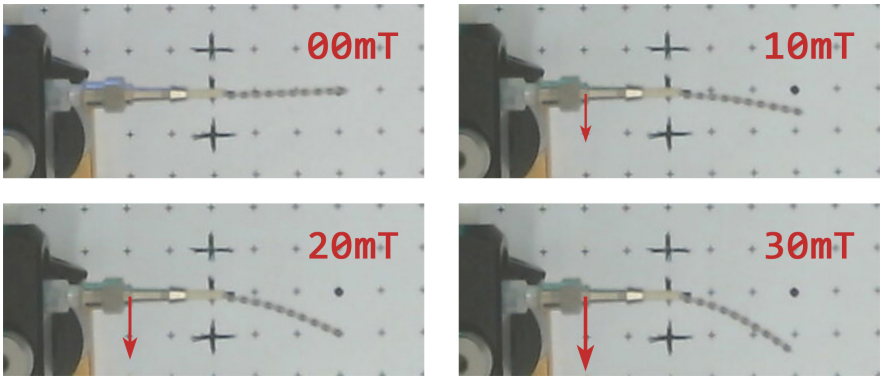


Figure 14: Example images of the bending tests. Here, the tip containing 10x1 magnets is shown in different fields (red arrow) strengths

Bending Radius Test: Wire Diameter After having determined the optimal magnet configuration, tests on wire diameters could be conducted. Three differently sized Nitinol wires [8] were used: 0.15 mm, 0.20 mm, and 0.25 mm. A tip extension, containing 1x10 magnets was built for each of the wires. These newly constructed tips were the subjected to the same bending tests as described before. Through these tests, it was determined that the 0.15 mm variant provided the best possible bending radius. While the tip may seem almost too flexible, the silicon coating, which is applied later, will add back a substantial amount of stiffness, which must be taken into consideration.

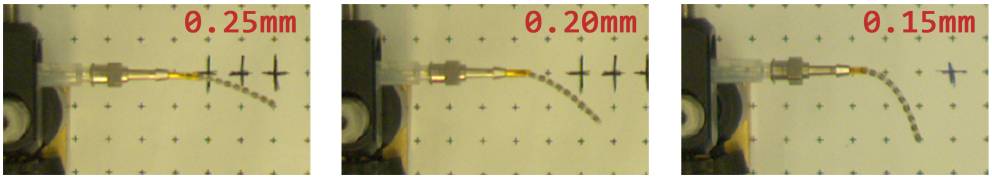


Figure 15: Three different wire diameters in a perpendicular 30mT magnetic field

Test in Model Following those initial tests a definitive tip extension containing 10x1 magnets and 0.15 mm Nitinol wire was constructed and coated with silicone. After attaching it to the catheter, first practical tests could be conducted inside of the abstract phantom (refer to section 2.2). During those tests the catheter was clamped into an existing advancer-unit and its tip was placed inside the phantom. Everything was then placed within the Navion-workspace. Using a computer connected to both the Navion and the advancer, the catheter could be fully controlled remotely. Movement patterns like advancing straight ahead, tight curves and maneuvering around obstacles were tested inside of the abstract model. It was revealed that the catheter would often kink right were the extension had been attached to the catheter, when entering sharp curves.

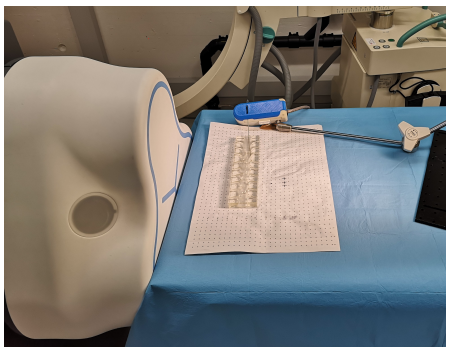


Figure 16: The test setup in front of the Navion

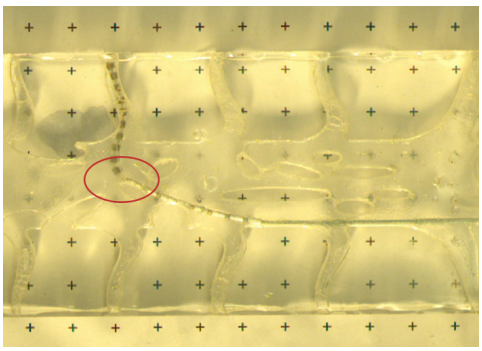


Figure 17: Shows the catheter sharply bending

Final Version To mitigate this problem and reinforce the problematic part, the magnet layout was slightly altered for the final version: the first magnet was directly attached to the catheter and the gaps were chosen to incrementally increase towards the tip of the extension. This would give the tip more strength, where it sharply bent before, while still allowing for enough flexibility to bend in a tight radius. As further testing in the 2D model confirmed, this improved the behavior of the implant while maneuvering.

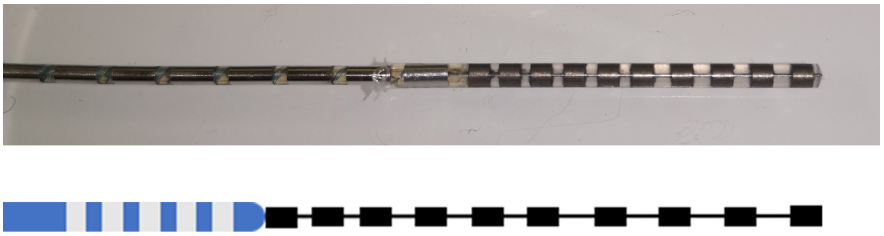


Figure 18: The final version of the modified catheter with silicone coating

Overall, the modification enabled magnetic steering of the catheter with reasonable bending radii. Even the dorsal root ganglions could be entered with careful steering. At the same time, all physical restrictions were satisfied. However, the electrodes proved to be somewhat problematic: they are very stiff. Often during experiments the tip could enter very tightly curved spaces but the electrodes were not able to follow. Building a totally customized catheter with more flexible electrodes and well integrated magnets could be beneficial.

4 Advancer Units

4.1 Theory

During the surgical implantation of an SCS-implant, the catheter has to not just be steered left and right, but also needs be moved along the spinal canal rather precisely. In a classical procedure this is accomplished manually by a surgeon. They can push and pull the catheter by its protruding end. For a truly autonomous system however, a computer has to be able to move the catheter in the same fashion. So called "advancer-units" are used to facilitate this longitudinal motion of the catheter by using electric motors. Luckily, existing advancer-units could be used. Only a few small modifications had to be performed, which are discussed in this section.

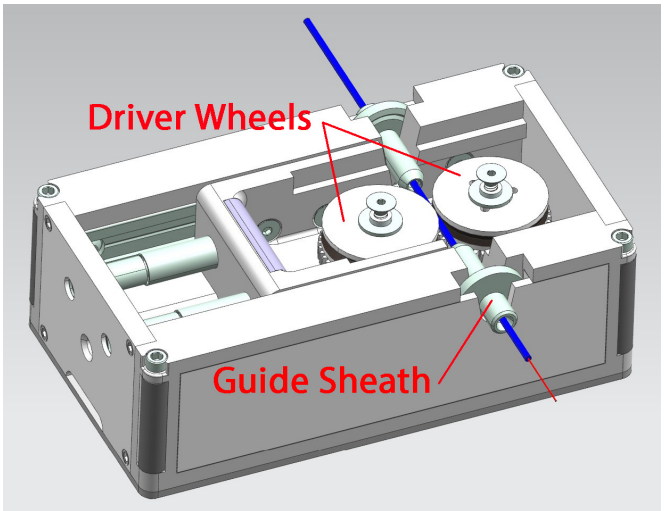


Figure 19: The existing modular advancer-unit in Siemens NX 12

Existing Units The existing advancers which were used are a design from the MSRL-lab at ETH Zürich and were initially constructed by Sean Lyttle (see figure 19). They feature two exchangeable wheels which are able to clamp various catheters. A heavily geared-down electric motor can precisely turn one of the wheels, which results in linear motion of the catheter. Thanks to a very modular construction, these units can be used for different catheter diameters: both the driver wheels as well as the sheath-guides can be replaced and adjusted to fit different catheter models. These were therefore the only modifications that had

to be carried out, to make the existing advancers work with the Avista™ MRI catheter from Boston Scientific [6].

Layout of the new System For the Avista™ SCS-implant two advancer-units should eventually be used: one will control the movement of the catheter itself. A second unit can insert and retract the guide wire, which runs throughout the whole length of the implant. With this setup, it is possible to automatically control the flexibility of the catheter tip by slightly retracting the guide wire.

The relative positioning of both units with respect to each other is very critical. They should be able to carry out their individual actuation without influencing each other. This is achieved by fastening them together with a hinge and looping the catheter in between the units. When the catheter is moved by the first advancer, excess length is just "stored" in the loop and the second unit can still hold onto the end of the catheter and control the guide wire undisturbed.

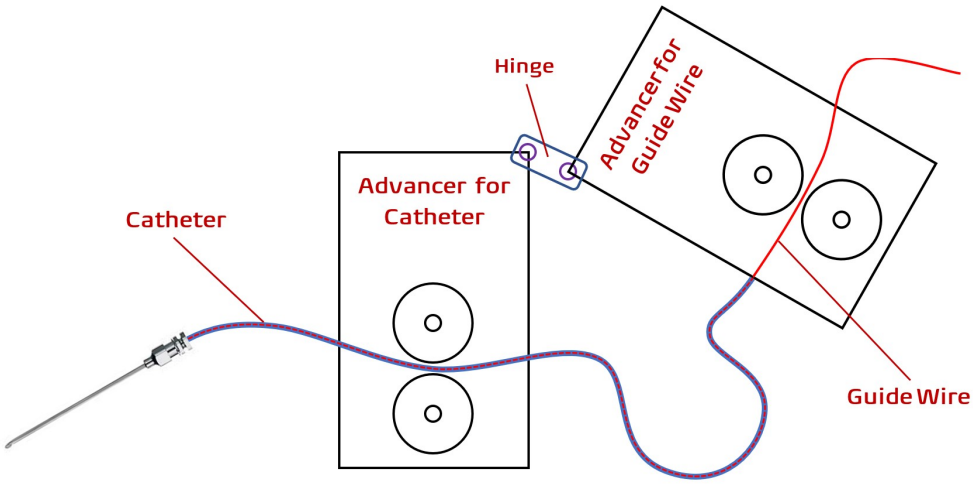


Figure 20: Shows the layout of both advancer-units

The absolute placement of this whole assembly however, is not very critical. The catheter advancer should simply be situated close by the opening of the Tuohy needle and will be held by a flexible arm. Ideally, the gap between the sheath of the catheter-advancer and the Tuohy needle is as small as possible, to avoid buckling or movement inconsistency due to elasticity in the catheter.

4.2 Requirements and Design

The only modifications undertaken, concerned the driver wheels and the sheath-like guides. Any other parts were kept as originally designed and manufactured.

Modifictions to the Catheter Advancing Unit For this component, first the driver wheels were were modified in the existing CAD-file: they feature a triangular groove to accommodate the catheter. This groove was adjusted to fit the size of the catheter. Additionally, the through-holes in both sheaths were designed to be slightly larger than the outer diameter of the catheter. One of the guides was also extended with a model of a female Luer-lock. This would allow for a tube to be attached, which can connect the advancer and the Tuohy needle. Such a tube can stabilize the catheter and prevents any fluids from leaking out of the Tuohy needle opening (refer to figure 21).

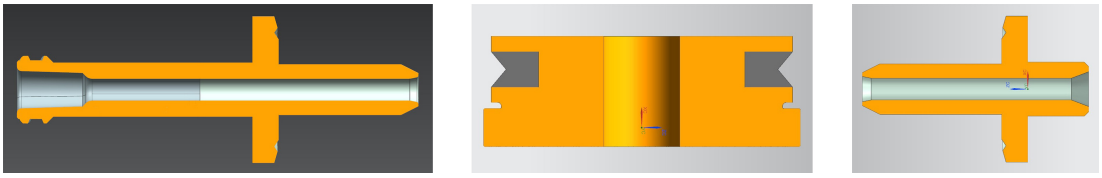


Figure 21: A cross-section through the parts that were modified (from left to right): the sheath with a Luer lock - driver wheel - exiting sheath

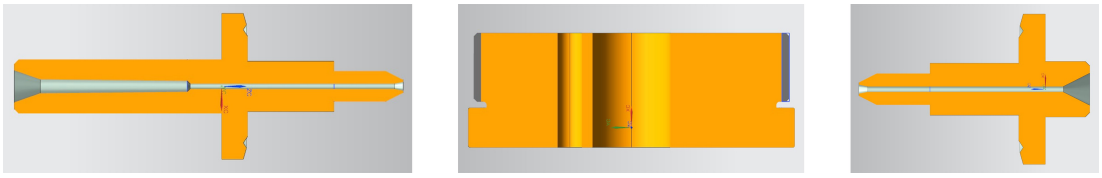


Figure 22: A cross-section through the parts that were modified (from left to right): the sheath with with conical catheter clamp - driver wheel - exiting sheath

Modifictions to Guide Wire Advancing Unit The sheaths and driver wheels for this unit were both modified to fit a guide wire with a diameter of 0.36 mm. The only special feature is an additional sheath extension which can hold onto the end of the catheter. The trough-hole of the first sheath has a slightly conical extension into which the catheter can be press-fit. Its purpose is to securely grip the end of the catheter and therefore enable precise movement

of the guide wire by cancelling any movement of the catheter coming from the other unit (refer to figure 22.

4.3 Materials and Methods

After the initial design of the modifications in Siemens NX had been concluded all those parts were exported as STL-files. They were then sent to a Polyjet 3D-printer; a special type of Selective Laser Sintering (SLS)-printer which enables multi-material printing. All sheats were printed using a rigid plastic while the wheels comprised both flexible and rigid plastic. The main body of the wheels is made of hard plastic. Only the contact area, where the catheter or guide wire is clamped, is made of flexible rubber to increase friction (grey colored areas in figure 21, 22). After printing, the gel-like support structure had to be removed from all parts. This was done by first blasting them with a high-pressure water jet and then rinsing the parts in a Hydroxide solution. After being left to dry for at least 24h the newly manufactured pieces could be installed on the advancer-unit

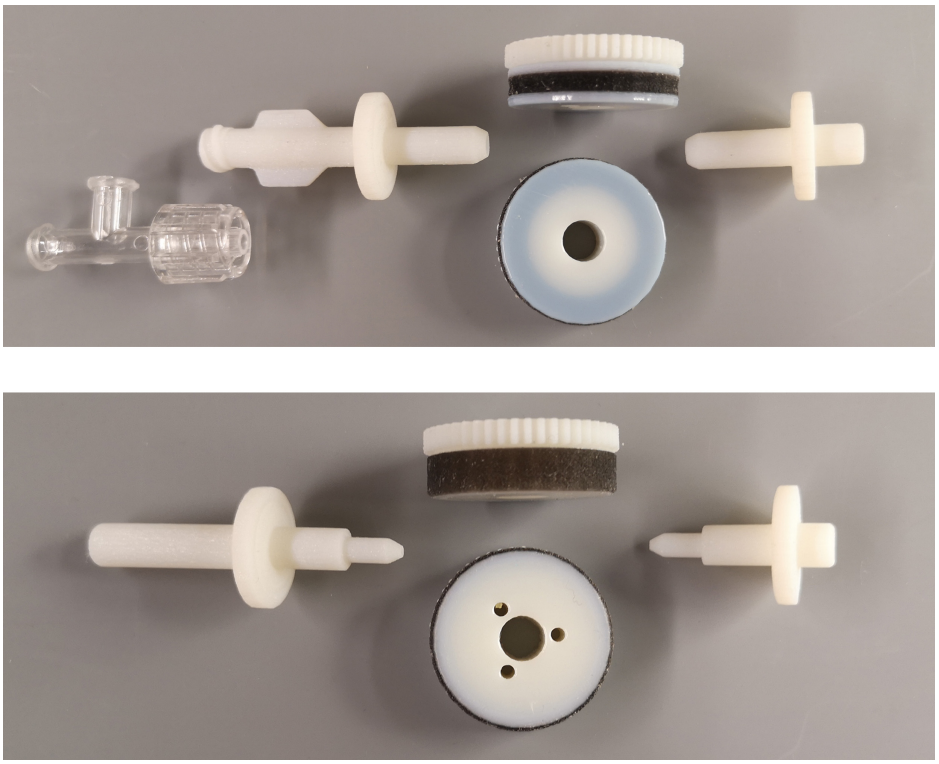


Figure 23: All the finished Polyjet-3D-printed parts

4.4 Results and Discussion

Due to time constraints, the modified advancers could not be used and tested in any of the final experiments. Although all customized parts were successfully manufactured, there was not enough time left to integrate the advancer-units into the software and ROS-framework. Nevertheless, the sheaths could be examined and both, the press-fit catheter mount, as well as the added Luer lock proved to work exactly as intended.

5 Software Implementation

5.1 Theory

So far, all necessary hardware components have been established: by using the Navion and advancer-units, a modified catheter can now be actuated from a remote computer. Controlling the magnetic fields manually (with joystick inputs) however, is not very intuitive and requires a fair amount of coordination, especially when also advancing the catheter at the same time. To mitigate this issue and further simplify the procedure, a control algorithm will be designed to manage those tasks. Ideally, such an algorithm is closed-loop, meaning that it will not only control all movements of the catheter, but also observe its position through the fluoroscope at all times. this section will discuss the full process of implementing such a control algorithm using the Robot Operating System (ROS)-framework.

Robot Operating System (ROS) A control-software of this sort requires a computer to interact with totally different hardware and software entities at the same time. For this reason, ROS was used as a framework to realize the controller. It is an open-source software, that is used to facilitate communication between various sensors, actuators, controllers, and scripts. A speciality of ROS is that the individual components do not have to be ROS-endemic. They are simply embedded and lightly "wrapped" by a ROS-framework. Thanks to ROS, for example, a motor and a Python script can be treated in very much the same way from the coders perspective.

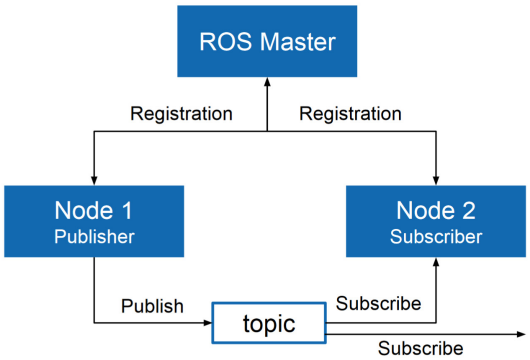


Figure 24: A schematic of the typical Node-structure in ROS [13]

Every single piece of software and hardware is represented as a "Node" in ROS. These Nodes can then exchange information and communicate through sending messages. When a node wants to send a message, it publishes it to a so called "topic", which is basically a message channel. Other nodes can then subscribe to those topics and retrieve the messages. Conveniently, ROS-nodes can also be scripts written in Python or other languages like C++ [13].

Outline The basic structure of the controller comprises two central tasks: a Tracker-Node (refer to section 5.2) visually extracts information about the catheter from the fluoroscope image in real time. The output from this Node are parameters like position, angle, and bending radius of the catheter tip. Additionally, some registration markers will also be read by this routine to gather information about the Navion-workspace. A second Controller-Node (refer to section 5.3) will process these extracted parameters and tries to steer the catheter along a predefined trajectory. To achieve this, the second node can set a desired magnetic field direction and strength, as well as advance the catheter back and forth.

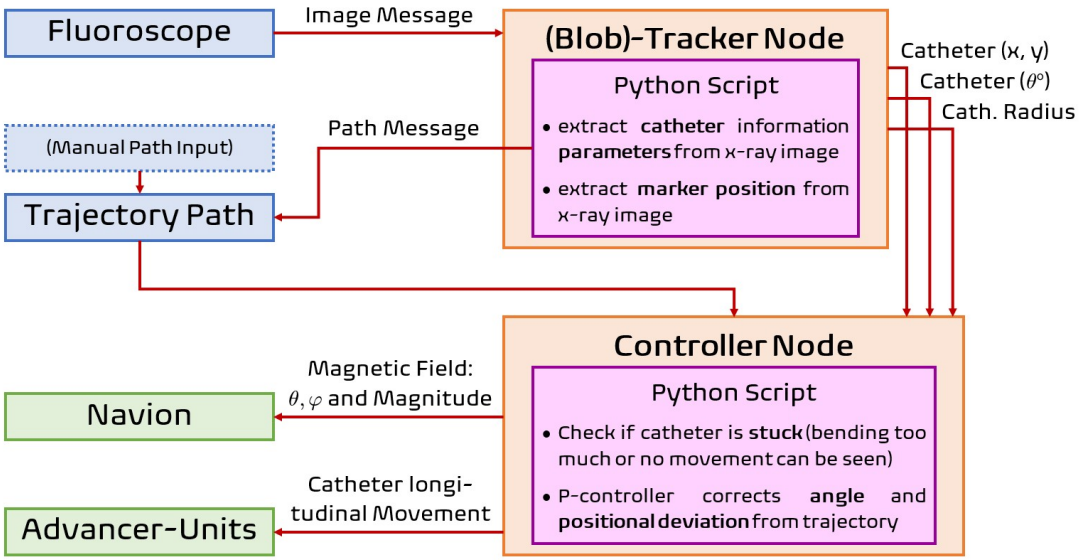


Figure 25: The structure of the software implementation in ROS

Both nodes each comprise one Python script to execute their task. Python was chosen because of its well established support for computer-vision libraries in addition to being very easy to implement in ROS. All the other parts of the

system, like the Navion, the advancer-units, and the fluoroscope machine are represented as distinct nodes as well, in order to be able to communicate with the Python scripts trough ROS-messages.

5.2 Tracking Node

This Node, or rather its Python-script (refer to section A.1), receives the x-ray image as input. The trackers purpose is then to retrieve the following information from a raw image:

- The catheter location (x and y coordinates), its tip angle (θ) with respect to the spine, and lastly the catheter tip curvature.
- Copper markers are placed on the model to outline the epidural space. From those, a path can be calculated, along which the catheter is supposed to move.
- Another set of copper markers indicate the Navion workspace-origin.

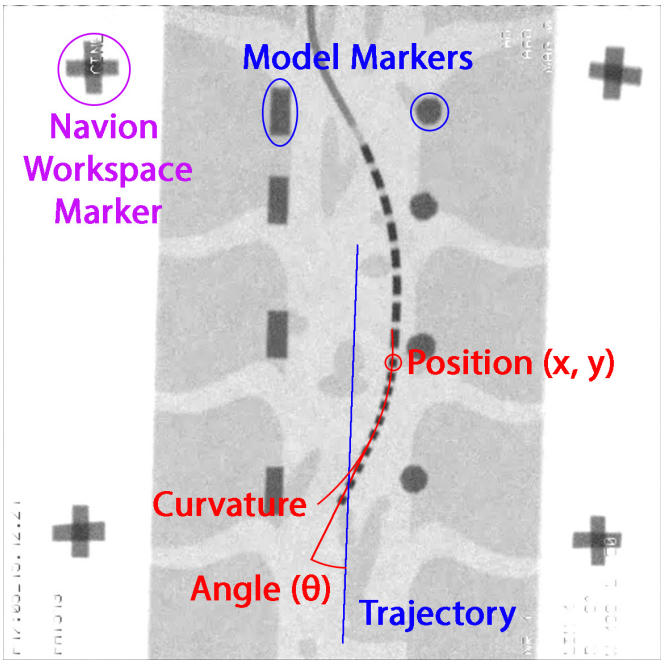


Figure 26: All parameters and markers are labeled in this x-ray image

To extract all this information from the image, a so called "blob tracker" is used multiple times. A blob tracker essentially recognizes connected patches of

similar colors, or in this case similar shades of grey. When applied to any input image, it will first recognize thousands of blobs with a wide variety of shapes and sizes. Fine tuning is then necessary, to obtain only the desired output set of points. This is done by filtering for patches of a certain size or shape through a couple of parameters within of the tracker.

OpenCV Blob Tracker Conveniently, blob trackers are a widely used tool in computer vision, therefore an existing implementation could be used. The OpenCV Python library provides a very user-freindly blob tracker module which was used as a foundation for the tracking node [34]. When fed an input image, it extracts all color-patches and provides a "keypoint" for every blob. This keypoint comprises the coordinates, size and orientation of the patch. The following parameters can be used to filter only for a desired set of color-patches [27]:

- "filterByArea": define a minimal/maximal area for blobs to be considered.
- "filterByCircularity": ranges from 0 to 1. A circular blob has a value of one, while a line is close to zero.
- "filterByConvexity": ranges from 0 to 1. The value is essentially the ratio of a the area of a blob divided by the area of its convex hull. Shapes like regular polygons score close to one, while a form like the letter "C" would score closer to zero.
- "filterByInertia": ranges from 0 to 1. Describes the "aspect ratio" of a blob. A square has an inertia ratio of one, while an infinitely long line has a value of zero.



Figure 27: A visual representation of all four blob-parameters

If all these parameters are carefully tuned, such a blob tracker can actually recognize very distinct shapes in an image while ignoring all other blobs. For

example, only crosses of a certain size or only perfect circles can be extracted from a given image. This adaptability was used to identify the aforementioned markers in the x-ray image. As long as all markers feature distinct shapes they are easily distinguished by a blob tracker. The catheter itself also can be recognized as very regularly spaced and roughly circular blobs forming a continuous chain. For each element a separate blob tracker was set up and was individually tuned. Their parameters and methods are described in the following paragraphs.

Model Marker Recognition At all times, the algorithm has to know where the spine or rather the phantom is located. If, for example, the patient were to move slightly during the procedure, the algorithm should compensate for that movement. Markers are consequently placed on the model, to the left and right side of the epidural space. The left side features perfect circles, while the right side is marked with long rectangles.

The circles are easily recognized by looking for almost perfect circularity, full convexity and an inertia ratio of about 1. Rectangles on the other hand, are quite different: they have the same convexity of 1 but at a lower circularity and only roughly 0.1 inertia ratio.

Once all the coordinates of these markers have been extracted, the path is calculated by simply interpolation a line through all the keypoints. Since the spine can barely bend over the span of three to four vertebrae, a linear fit is precise enough. By using the difference of the left and right markers the patients orientation with respect to the Navion can be determined.

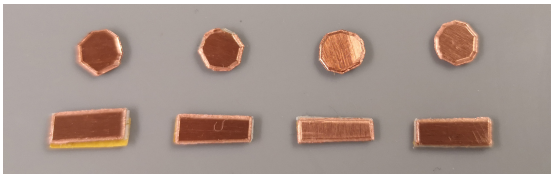


Figure 28: Copper markers for recognizing the models orientation

Navion Workspace Marker Recognition In addition to knowing where the spine is, the algorithm also needs information about the Navion workspace or rather its origin. This is necessary, because eventually the controller will set the magnetic field to a certain value. This value cannot be generated homogeneously

inside the whole workspace, therefore it is simply requested at the tip of the catheter. For this reason, the controller needs to precisely know, where the catheter tip is situated not only in the picture, but with respect to the physical Navion workspace.

Four cross-shaped markers are placed on the corners of a square at 4 cm distance to the Navion workspace origin. These markers can be easily identified by the blob tracker: their inertia ratio is nearly 1, similar to a circle, but they are the only markers to feature a convexity significantly below 1. By simply averaging the coordinates of all these cross-shaped markers, the Navion workspace origin is determined.



Figure 29: The copper markers for the navion workspace origin

Catheter Feature-Recognition The catheter-tracking itself is substantially more intricate than tracking the markers. While the magnets and electrodes provide a fairly decent contrast to the surrounding tissue in the fluoroscope, their shape can vary quit a bit due to noise. In addition to that, noise with roughly the same shape as catheter features can be present. Among this noise, the catheter has to be correctly identified.

First, a blob tracker is set up which filters for roughly circular and very small blobs. This tracker then predominantly yields keypoints belonging to the catheter. However, some small amount of noise is detected as well and cannot easily be filtered. Some noise-patches are just too similar to catheter features. Therefore, a subroutine had to be written to determine which of the keypoints are part of the catheter and which ones are not, after the pre-filtering. This subroutine is rather lengthy and accounts for a significant part of the Tracking-Node Python script (refer to section A.1). The catheter-recognizing algorithm seeks out one group of points in the image that loos the most like a catheter. Distinguishing features of a catheter were assumed to be:

- The magnets are spaced fairly closely and in very regular intervals.
- The catheter has no sharp bends; therefore, the angle from one point to the next should not change very much (in other words, the curvature should change very little).

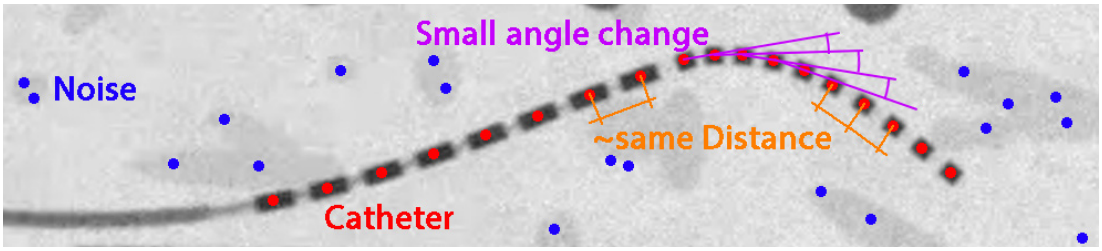


Figure 30: Both criteria illustrated in an x-ray image of the catheter

Both of those criteria are applied twice in two separate steps by the algorithm. First, a "starting point" which is certain to lie on the catheter has to be found: to do so, for each keypoint in the image a separate chain of points is constructed and stored in a list. These chains are created by starting at one point and then continuously looking for the next closest one and appending it to the list. Each sequence is then assigned a score according to how uniform the points are spaced and how little the angle between the points changes throughout the entire chain. A lower score means "more likely to be shaped like a catheter". The lowest scoring chain is then picked and assumed to be part of the catheter. If any catheter is present in the image, this best scoring chain is almost guaranteed to be part of if. Thus a starting point or rather starting-chain has been found.

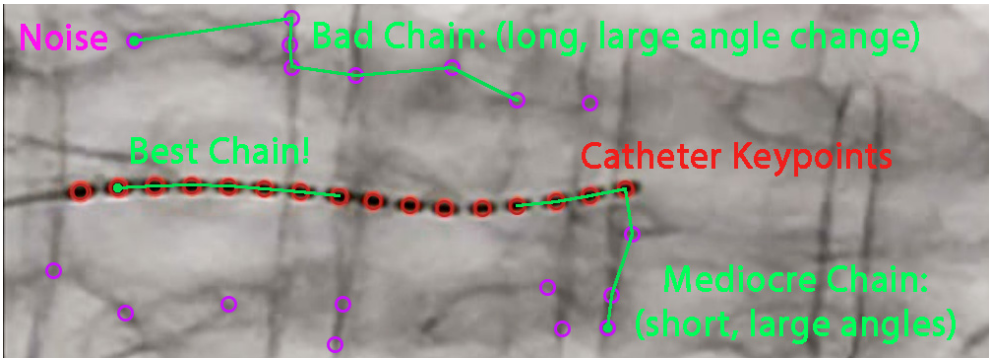


Figure 31: An image illustrating three examples of such initial chains. Normally, a chain of this sort is created for every single keypoint in the image.

In a next step, more close points are appended to this initial chain one by one. A recursive function always "expands" the previous sequence by the next, most fitting point. Again, the new point is selected by the same angle and distance criteria as before. This expansion function is terminated as soon as there are no more fitting points present in the image, that are not yet part of the catheter chain. All other points are discarded as noise.

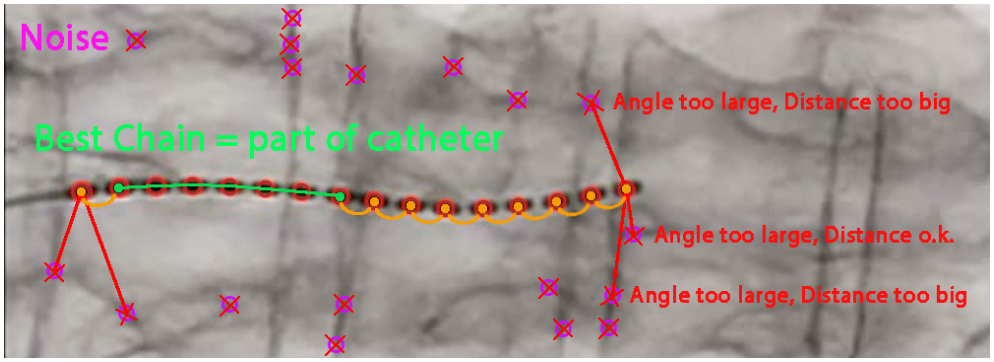


Figure 32: An illustration of how the initial chain is expanded

Only once the full catheter has been discovered, are its parameters determined: the x and y coordinates of the tip, the angle at which the tip is tilted, and lastly the curvature of the whole tip-section. The position is chosen to be some point of the catheter roughly at the connection point of the tip and the catheter electrodes. The angle is calculated, by fitting a first order polynomial through the front few points of the catheter and is defined with respect to the path trajectory. The curvature can only be calculated "descretly": all angles between the points lying on the tip are added together and are then averaged.

Output Since all these parameters could be not only of interest during the surgical procedure but are also essential for debugging purposes, they had to be somehow visualized. A graphical overlay was therefore imposed on the original x-ray image with the help of OpenCV functionalities. This image was the published as a ROS-message and could be visualized in an ROS Visualization (RViz)-panel in real time.

Finally, all the parameters are passed on to the control Node. A ROS-message containing all the information is created and published to its corresponding "catheter_info" topic. The controller-Node can then read it and process the message.

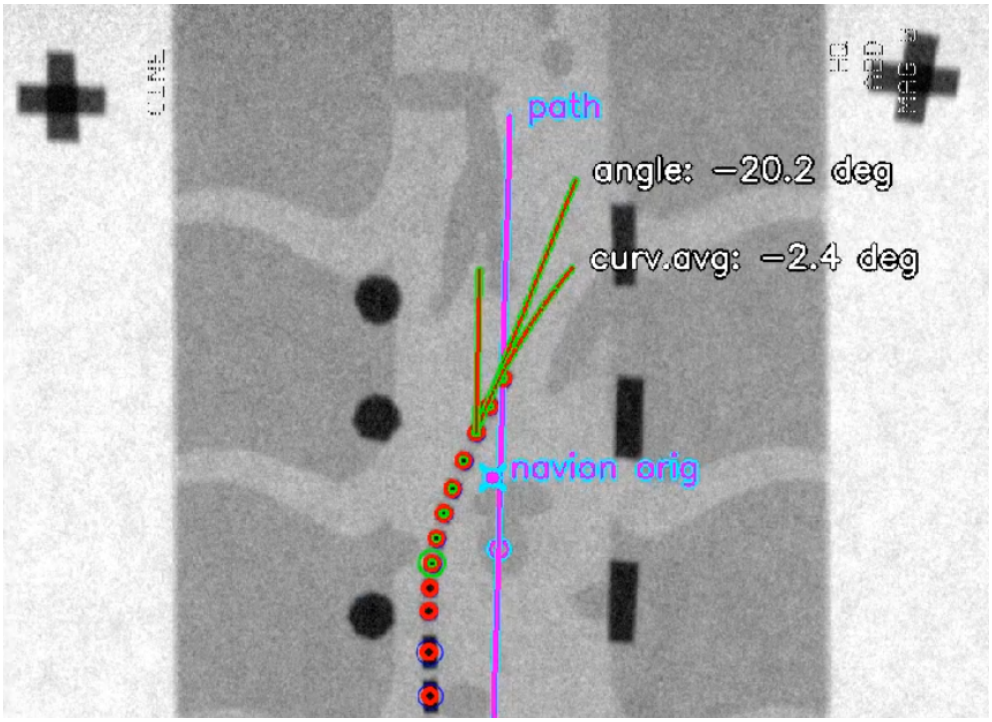


Figure 33: All this information displayed with an overlay in the original x-ray image

5.3 Control Node

The control node is isolated and does not directly receive the x-ray image. Its only input is the information about the catheter location, angle, and curvature, as well as all the marker references. The core purpose of this control node, is to implement a simple proportional controller which will adjust the magnetic field, so that the catheter is steered along the trajectory. However, there are multiple ways in which the catheter can get stuck and they have to be detected by the controller as well.

Error Decision-Logic In its most basic form the control node will continuously check if the catheter is stuck or any other error is present. Only if this is not the case, will the controller advance the catheter. There are two main ways, in which the implant can get stuck on obstacles. They were observed in initial manual test on the abstract phantom:

- The catheter is pushed against an obstacle and buckles. In this case, the curvature of the tip clearly exceeds an otherwise low value.

- On much rarer occasion, the catheter tip can get stuck due to an increase in **friction** when moving between tight obstacles. In that case, the tip is still **straight**, but will not move anymore.

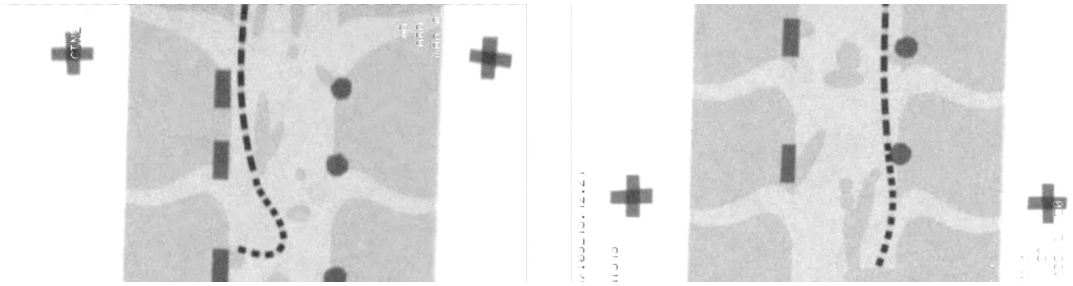


Figure 34: An Image showing the catheter being stuck in a bent position (left) and in a straight position (right)

To detect a possible error, the control algorithm constantly checks for two properties, before moving the catheter further: for one, if the curvature of the tip exceeds an experimentally evaluated threshold, it is immediately clear, that an obstacle was encountered. Since the magnetic tip cannot describe such tight radii by itself, this error can be easily differentiated from a case where the tip is only bent due to the magnetic fields.

On the other hand, the controller can constantly compare the input it gives to the advancer with how much the catheter tip actually moves. If those two values strongly differ, or the tip does not move at all, an error is present as well. It means that the advancer is running but not actually moving the catheter anymore. Instead it is just buckling somewhere in between the electrodes and the advancer-unit.

As long as none of those criteria are met, the controller executes its "proportional controller" function (refer to section 5.3 "Proportional Controller"). But as soon as a non-traversable obstacle is encountered, and at least one of the error criteria is met, the catheter is backed up a bit and the "Evasion" function is executed (refer to section 5.3 "Evasion").

Proportional Controller The purpose of this function is to simply steer the catheter along a linear trajectory. Hereby, the distance between the trajectory path and the catheter tip is the controlled variable. This "error-distance" should be minimized by the proportional controller. To achieve that however, the

controller needs an actuating variable: in this case it is the direction of the magnetic fields created by the Navion. Since the tip of the catheter roughly follows the magnetic field direction, the P-controller can govern the catheter tip angle and steer it to the left or right side.

The actuating variable is calculated by multiplying the error-distance with a constant P, which is the very definition of this type of controller. By increasing or decreasing the value of P, the catheter is regulated back to its path more or less aggressively. Unfortunately, due to the nature of a P-controller, the deviation from the target can never be fully eliminated (static tracking error) [16].

$$\text{(field angle) } \alpha = P \times \text{error-distance}$$

Evasion This function is called, whenever an obstacle is encountered. After an error has occurred, it will first try to move the catheter backwards until it sees it visually doing so. This confirmation is necessary because sometimes an unknown length of catheter can be "stored" in buckling bends. Afterwards, the function slightly corrects the trajectory to the left or right and tries to advance again. Ideally the obstacle can be circumnavigated by doing so. However, since the controller has no knowledge of the obstacles depth and shape it cannot know how far the catheter actually has to be backed up. For this reason an iterative approach was implemented: after having backed up once and still hitting the same obstacle, a second reversing maneuver is initiated. Each time the backup distance is increased slightly, so that eventually the catheter will be able to avoid the obstacle.

5.4 Results and Discussion

Testing the components of this software was unequally difficult. The blob-tracker node could be tested extensively, isolated from the rest of the code. On the other hand, testing the controller required an already working Tracker-Node as well as the full physical setup with the Navion, a phantom, and the catheter. Most of the conclusions for the Controller-Node were therefore only drawn from the final experiment discussed in section 6.1.

Tracking Node The tracking node could be tested almost live and even during coding. To do this, without using a real fluoroscope, a video of the catheter was pre-recorded through the fluoroscope. This video was then published by an Emulator-Node (refer to section A.3) to generate the ROS-messages that would otherwise come from the real fluoroscope. Thanks to being able to test this Node almost indefinitely, all the tracker-parameters could be tuned and optimized.

However, during the final tests when the real fluoroscope was used again, the image quality changed significantly and more noise was present. This required re-tuning the blob tracker parameters by trial-and-error to match the new conditions. In general, the OpenCV blob-tracker is rather susceptible to image brightness and noise. Having a user-interface, to quickly tune the parameters would be helpful and could lessen the time required to set up an experiment.

The catheter-recognition part of the Tracking node worked almost flawless in all experiments. As long as the preceding blob-tracker yielded enough keypoints, this routine was able to consistently find the catheter and read out its parameters. Even heavy noise or missing points will not cause any disruption. Although, the required computational power was observed to increase if more noise is present. Nontheless, even a standard laptop was able to perform the tracking operations on a 30 fps video which is more than enough for the fluoroscope framerate of 8 Hz.

The whole tracking node was integrated in an RViz-panel and can be conveniently opened with a single command. This proved to be useful during experiments and very user friendly. Additionally, any information gathered about the catheter can be visualized nicely in real time.

Controller Node The controller node proved to be much more sensitive than the tracker during the experiment. While the basic controller logic could be tested and proved to work in principle, a lot of room for improvement remains. The controller could only be tested once: during the final experiment. The code had to be prepared mostly in theory with no possibility to practically test the controller before the experiment. This led to a lot of parameters that were tuned incorrectly, for example, the controller proportionality P or the advancer motor speed.

During the experiments, the control algorithm was still able to successfully manipulate the magnetic fields and the advancer unit. In addition to that, one or even two obstacles could repeatedly be circumnavigated automatically. Unfortunately, the connection between the Navion RViz-interface and the controller was very unstable and would often crash for an unknown reason.

During the final experiment some of the variables were adjusted but quite a lot of fragility remained. Overall, the controller is not especially user-friendly and has multiple aspects with much room for improvement:

- Through more practical testing, parameters such as controller strength P, motor speeds and detection thresholds can be optimized a lot.
- Having a user-interface in RViz to start, stop and manually override the control algorithm would improve user-friendliness by a lot. Additionally, outputting debugging messages of the controllers error decision logic could be helpful in further development.
- Implement a Proportional-Integral-controller to eliminate the static tracking error.
- Even though, the controller error logic itself works in principle, it conducts a very naive "trial and error" approach. Adding a self-adjusting parameters to provide some memory over previously encountered obstacles or even a machine-learning approach could make the controller a lot more efficient.
- The physical setup has a lot of slack, especially in the advancer-unit and due to the flexibility of the catheter. Reducing this looseness would permit the controller to work more precise. Otherwise a lot of positional uncertainty has to be taken into account which consequently results in a less responsive controller.

6 Experiments

This section only covers the final tests of the control algorithm, since they involve every other part as well. Test for the individual components can be found at the end of their respectable chapter.

6.1 Full Controller Test in Abstract Phantom

6.1.1 Motivation

After having only been able to test the individual parts of this project is was paramount to bring everything together and test it at once. While components like the Tracking-Node or the the phantoms could be tested individually, that was not the case for the control algorithm. It can only be tested in the full physical setup. Additionally, in reality even individually working parts can sometimes cause trouble once they are connected to other elements. It is impossible to think of all potential issues beforehand, therefore a test on the full system was planned since the beginning. Especially the communication between the Controller-Node and the Navion an fluoroscope machine had to be tested.

6.1.2 Experimental Setup

For this experiment the final version of the modified catheter (refer to section 3.4) was used and placed inside of the abstract phantom with moderate obstacles (refer to section 2.5). The model was then filled with the 1.2 wt% gelatin mixture. Everything was placed centered in the Navion workspace. Copper markers made of 1 mm thick Copper sheet stock were then attached to the model and the table. Since the proprietary advancers were not yet implemented and existing unit, which is part of the Navion, was used. This meant that the guidewire could not be actuated remotely and was therefore left slightly rectracted (about 2 cm) With this setup multiple runs of the control algorithm were performed.

Before each run the catheter was positioned at x-ray image boundaries. Then, the Tracking-Node, the fluoroscope and the Controller-Node had to all be started at roughly the same time. This had to be done manually. The catheter was given time to run through the whole frame of the fluoroscope image which usually involved circumnavigating one or two larger obstacles. This course of actions was

repeated about 15 times. All attempts were recorded including the magnetic field display.



Figure 35: The experimental setup

6.1.3 Results and Discussion

The system succeeded multiple times at navigation the catheter from one end of the x-ray image boundaries to the other while bypassing one or two larger obstacles. The communication between all Nodes was working well enough for the experiments to be carried out.

Phantom and modified Catheter Both the abstract phantom and the catheter worked as intended. However, containing the gelatin inside of the model proved to be challenging. Tape was used for sealing any openings but it did not stick to the silicone very well. Potentially, a 3D-printed encasing for the whole phantom could serve as a barrier to contain the gelatin.

Tracking-Node This part of the system was surprisingly reliable once the blob tracker parameters had been adjusted to the live fluoroscope image. It constantly succeeded at recognizing the catheter when it was visible in the image and almost never lost track of it. The RViz-panel in which served as an interface

for this Node worked perfectly and could be run and shut down with only one command from the terminal.

Controller-Node The Controller-Node on the other hand was not very user-friendly. Due to unknown communication errors between it an the Navion RViz interface it had to be restarted for every single run. Additionally, since the controller was never tested before, motor speeds of the advancer were rather slow. When just following the trajectory without any obstacles the P-controller function revealed quite a significant static tracking error. This could be partially due to the catheter being not very responsive in the rather thin gelatin mixture used in this experiment.

Other Elements The Navion has an existing interface and was therefore easy to use. Unfortunately, its coils and motor controllers overheat within two to three minutes when running at the required field strengths for this experiment. This meant that after every run the system has to be given a few minutes of time to cool down before it is ready again.

Similar waiting issues were present with the fluoroscope. It has a runtime limit of five minutes after which it automatically shuts down. This limit has to be manually reset whenever it is reached. Some runs were interrupted by this timer and could not be completed.

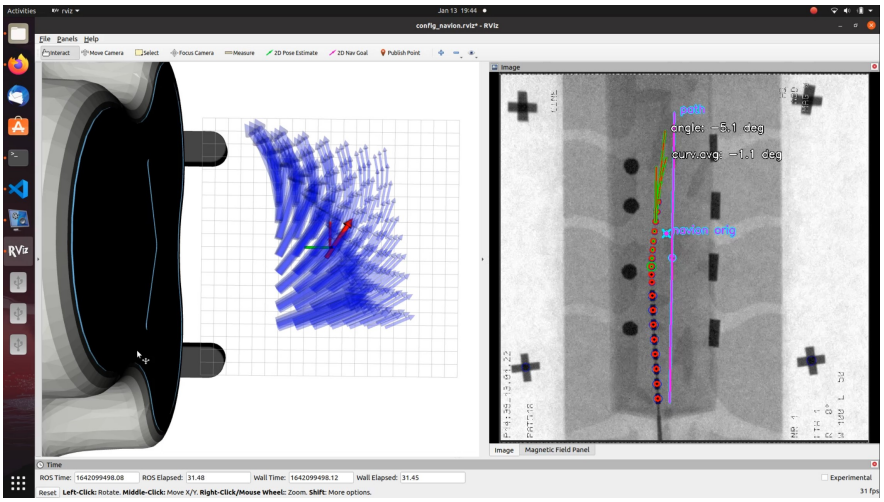


Figure 36: The user interface during the experiment showing the magnetic fields and the tracker-overlay

7 Conclusion and Outlook

7.1 Conclusion

The original goal of this project was to improve upon the Spinal Cord Stimulation procedure by eliminating some of its flaws in connection with the manually steered catheter. While no mature control algorithm could be developed, which would totally autonomously conduct the surgery, invaluable insights for further development of such an automated SCS system were acquired nonetheless. The individual components which are necessary for improving the SCS procedure were developed successfully.

Multiple highly useful phantoms for testing the setup, as well as the magnetically modified catheter were finished. These parts have been extensively tested and proved to be usable. They made testing the software part possible and provided practical means for carrying out a rudimentary version of the SCS procedure in a lab environment.

Additionally, a reliable and user friendly visual tracker was developed for extracting information about the catheter from an x-ray image. This tracker was also tested on multiple occasions and worked flawless during the concluding tests of this project.

Finally, a simple version of a control algorithm was successfully implemented in Robot Operating System. It served well to connect all the individual components involved during the SCS procedure. Concluding tests could be carried out and a catheter was autonomously maneuvered through an abstract phantom of the epidural space. Albeit this controller being more of a proof-of-concept its core logic proved to work and it serves as a good foundation for further development.

Creating, testing, and optimizing the full autonomous SCS system is an ambitious task for the scope of a Bachelors Thesis. Therefore a few things had to be left out due to time constraints. Notably the full 3D model of the epidural space, as well as the customized advancers were not completed. Still, the foundation for those components was laid out. Especially the 3D phantoms will be essential for testing, once the whole system is more optimized.

7.2 Outlook

Particularly because there are a few unfinished part to this project, this section seeks to give some suggestions on what could be improved during further development of such a system. Many of the valuable observation made during testing could not yet be implemented and are therefore summarized here.

Epidural Space Phantoms While the abstract models were useful enough, eventually, having a fully detailed reconstruction of the epidural space is necessary. Some 3-dimensional features like indentations from the vertebrae could be a possible obstacle for the catheter. Testing in such models is the next step in being closer to the real procedure.

Catheter Modification The modifications described in chapter 3 were sufficient for magnetically steering the catheter. However, the electrodes are quite stiff and the magnetic tip adds a significant amount of excess length to the implant. This was sometimes impractical during test and might not be suited for actual implantation. For these reasons, building a custom catheter which nicely incorporates the magnets could eventually be necessary.

Advancer Units While the catheter can well be controlled by existing advancers, such the one from the Navion, having two of them provides more control over the catheter: the guidewire can be actuated autonomously as well. This gives the control algorithm the possibility of adjusting the catheters stiffness. This ability could be useful for following tight curves while still being able to have a stiff catheter for piercing connective tissue.

Software Implementation Even though, the Tracker-Node worked fairly reliably during tests, it could well benefit from a user-interface to quickly adjust the blob tracker parameters to changes image quality.

The Control-Node on the other hand has a lot more room for improvement. First, by doing more testing its reliability could be improve significantly. Additionally, such a complex task might benefit from a more advanced control logic. Equipping the controller with some machine-learning capabilities would allow it

to do more intelligent path planing instead of just working in a trial-and-error fashion.

Another option that has been discussed, is giving the controller a scanned 3D model of the patients epidural space. A surgeon could then predefine a suitable path which avoids obstacles. The control algorithm would then focus more on precisely following that path instead of coping with the incredibly hard task of detecting obstacles in soft tissue.

Total System Overall, it might not even be necessary to fully automate the SCS procedure to improve it. Highly likely, a surgeon will have to be present regardless during such an intervention. In that case, he might as well still perform the one task, that is incredibly hard to do for an algorithm: making nuanced decisions on complex topics, such as human anatomy, based on years of experience. Nonetheless, some supportive tasks might still be carried out by an algorithm.

What should, however, definitely be pursued and developed further is the aspect of being able to remotely control the catheter. This is a true improvement to the traditional SCS procedure since it facilitates much more direct control over the catheter and does not require the surgeon to be present close to the harmful x-rays. This could improve the outcome for both, the patient and the surgeon significantly .

References

[1] Altropol. Kunststoff GmbH: NEUKASIL RTV 2 K Silicone. *https://www.altropol.de/produkte/neukasil-rtv-2-k-silicone/*, (Date Accessed: 2021-12-15).

[2] Z. Baber and M. A. Erdek. Failed Back Surgery Syndrome: Current perspectives. *Journal of Pain Research*, 9:979–987, 2016.

[3] Q. Boehler. Navion - Magnetic Field Characterization. 2021.

[4] Q. Boehler. Navion Developper Guide. 2021.

[5] A. P. Boezaart, A. Prats-Galino, O. C. Nin, A. Carrera, J. Barberán, J. M. Escobar, and M. A. Reina. The Posterior Lumbar Epidural Space: Three-Dimensional Reconstruction of High-Resolution MRI: Real and Potential Epidural Spaces and Their Content in Vivo. *Pain Medicine (United States)*, 20(9):1687–1696, 2019.

[6] Boston Scientific. Avista MRI 8 Contact Lead Kit. *https://www.bostonscientific.com/en-US/products/spinal-cord-stimulator-systems/precision_montage.html*, (Date Accessed: 25-11-2021).

[7] Boston Scientific. Precision Spectra™ Spinal Cord Stimulator System. *https://www.bostonscientific.com/en-US/products/spinal-cord-stimulator-systems/precision_spectra.html*, (Date Accessed: 2021-12-14).

[8] Chamfr. Medical Device Wire. *https://chamfr.com/product-category/wire/*, (Date Accessed: 2021-12-18).

[9] J. Dahlhamer. Prevalence of Chronic Pain and High-Impact Chronic Pain Among Adults. *MMWR. Morbidity and Mortality Weekly Report*, 67, 2018.

[10] W. Deardorff. Types of Back Pain: Acute Pain, Chronic Pain, and Neuropathic Pain. *https://www.spine-health.com/conditions/chronic-pain/types-back-pain-acute-pain-chronic-pain-and-neuropathic-pain*, (Date Accessed: 2021-11-23).

[11] Digimation. Vertebral Column Spinal Cord 3D model. *https://www.cgtrader.com/3d-models/character/anatomy/vertebral-column-spinal-cord*, (Date Accessed: 2021-11-22).

[12] DocCheck Medical Services GmbH. Spinal Cord Stimulation. *https://flexikon.doccheck.com/de/Spinal_Cord_Stimulation*, (Date Accessed: 22-11-2021).

[13] R. S. L. ETZ. Programming for Robotics - ROS Course. *https://rsl.ethz.ch/education-students/lectures/ros.html*, (Date Accessed: 2021-12-19).

[14] D. Filgueiras-Rama, A. Estrada, J. Shachar, S. Castrejón, D. Doiny, M. Ortega, E. Gang, and J. L. Merino. Remote Magnetic Navigation for Accurate, Real-time Catheter Positioning and Ablation in Cardiac Electrophysiology Procedures. *Journal of Visualized Experiments : JoVE*, (74):3658, Apr. 2013.

[15] J. Grider, L. Manchikanti, A. Carayannopoulos, M. L. Sharma, C. C. Balog, M. E. Harned, V. Grami, R. Justiz, K. Nouri, S. M. Hayek, R. Vallejo, and P. J. Christo. Effectiveness of spinal cord stimulation in chronic spinal pain: A systematic review. *Pain Physician*, 19(1):E33–E54, 2016.

[16] L. Guzzella, C Küttel, and M. Reinders. Regelungstechnik 1 Zusammenfassung der Vorlesung (HS 2019), Kapitel 10.1 - 10.4. *ETH Zürich*.

[17] C. M. Hassan, P. Trakampan, and N. A. Peppas. Water Solubility Characteristics of Poly(vinyl alcohol) and Gels Prepared by Freezing/Thawing Processes. In Z. Amjad, editor, *Water Soluble Polymers: Solutions Properties and Applications*, pages 31–40. Springer US, Boston, MA, 2002.

[18] D. Hoy, L. March, P. Brooks, F. Blyth, A. Woolf, C. Bain, G. Williams, E. Smith, T. Vos, J. Barendregt, C. Murray, R. Burstein, and R. Buchbinder. The global burden of Low Back Pain. *Annals of the Rheumatic Diseases*, 73(6):968–974, 2014.

[19] H. Knipe and Y. Baba. Spinal Cord: Reference Article. *https://radiopaedia.org/articles/25075*, (Date Accessed: 2021-11-22).

[20] P. C. Y. Lim, J. J. H. Toh, J. K. X. Y. Loh, E. C. Y. Lee, D. T. T. Chong, B. Y. Tan, K. L. Ho, C. K. Ching, and W. S. Teo. Remote magnetic catheter navigation versus conventional ablation in atrial fibrillation ablation: Fluoroscopy reduction. *Journal of Arrhythmia*, 33(3):167–171, June 2017.

[21] B. Lundeland, M. Toennis, M. Züchner, L. Janerås, A. Stubhaug, and P. Hansson. Spinal cord stimulation for the treatment of peripheral neuropathic pain. *Tidsskrift for Den norske legeforening*, June 2021.

[22] L. Manchikanti and S. Atluri. chapter 152 - Lumbar Epidural Nerve Block. In S. D. Waldman and J. I. Bloch, editors, *Pain Management*, pages 1281–1293. W.B. Saunders, Philadelphia, 2007.

[23] A. Mashari, M. Montealegre-Gallegos, J. Jeganathan, L. Yeh, J. Q. Hiansen, M. Meineri, F. Mahmood, and R. Matyal. Low-cost three-dimensional printed phantom for neuraxial anesthesia training: Development and comparison to a commercial model. *PLoS ONE*, 13(6), 2018.

[24] A. S. Narain, F. Y. Hijji, K. H. Yom, K. T. Kudaravalli, B. E. Haws, and K. Singh. Radiation exposure and reduction in the operating room: Perspectives and future directions in spine surgery. *World Journal of Orthopedics*, 8(7):524–530, 2017.

[25] F. H. Netter. *Atlas of Human Anatomy*. 2019.

[26] R. B. North, D. H. Kidd, F. Farrokhi, and S. A. Piantadosi. Spinal cord stimulation versus repeated lumbosacral spine surgery for chronic pain: a randomized, controlled trial. *Neurosurgery*, 56(1):98–106; discussion 106–107, 2005.

[27] OpenCV Team. Blob Detection Using OpenCV (Python, C++). *https://learnopencv.com/blob-detection-using-opencv-python-c/*, (Date Accessed: 31-12-2021).

[28] A. Prats-galino, M. A. Reina, M. M. Haramija, A. Puigdellivol-sánchez, J. S. Molina, and J. A. D. Andrés. LSNA Virtual 3D Model of the Lumbar Spine (Manual). 2019.

[29] A. K. Rock, H. Truong, Y. L. Park, and J. G. Pilitsis. Spinal Cord Stimulation. *Neurosurgery Clinics*, 30(2):169–194, 2019.

[30] Ruth Lilly Medical Library and UI School of Medicine. Free Spine 3D Model. *https://sketchfab.com/3d-models/spine-39ad9be7118d402ca7266fab01b75fed*, (Date Accessed: 2021-11-22).

[31] M. S. M. Said and N. Seman. Preservation of gelatin-based phantom material using vinegar and its life-span study for application in microwave imaging. *IEEE Transactions on Dielectrics and Electrical Insulation*, 24(1):528–534, 2017.

[32] L. R. Sass, M. Khani, G. C. Natividad, R. S. Tubbs, O. Baledent, and B. A. Martin. A 3D subject-specific model of the spinal subarachnoid space with anatomically realistic ventral and dorsal spinal cord nerve rootlets. *Fluids and Barriers of the CNS*, 14(1):1–16, 2017.

[33] Seattle Science Foundation and N. Ramana. Lumbar Spinal Cord Stimulation Trial. *https://www.youtube.com/watch?v=GpAPwBnK-6w*, sea(Date Accessed: 24-11-2021).

[34] O. Team. OpenCV Computer Vision Library. *https://opencv.org/*, (Date Accessed: 31-12-2021).

[35] The Spine & Pain Institute of New York. Spinal Cord Stimulator Trial. *https://www.youtube.com/watch?v=-rWuzfYkd5M*, (Date Accessed: 24-11-2021).

[36] Universitätsklinikum Freiburg. Neuromodulation. *https://www.uniklinik-freiburg.de/schmerzzentrum/therapien-fuer-schmerzpatienten/neuromodulation.html*, (Date Accessed: 24-11-2021).

[37] K. L. Wininger. Radiation Exposure in Percutaneous Spinal Cord Stimulation Mapping: A Preliminary Report. *Pain Physician*, 1;13(1;1):7–18, 2010.

A Appendix

A.1 Tracking-Node source-code

```
#!/usr/bin/env python
# the actual service, doing the blob tracking

import numpy as np
from numpy.lib.utils import info
import numpy.ma as ma
import matplotlib.pyplot as plt
from numpy.ma.core import doc_note
import scipy
import scipy.ndimage
import random
import time

import rospy
import sys
import cv2
from cv_bridge import CvBridge
from sensor_msgs.msg import Image

from epi_tracker.msg import cath_info



# ================================================================================
# CODE main body ###############################################################
# ================================================================================



# CONVENIENCE definitions ========================================================
bridge = CvBridge()
black   = (000, 000, 000)
white   = (255, 255, 255)
blue    = (255, 000, 000)
green   = (000, 255, 000)
red     = (000, 000, 255)
cyan    = (255, 255, 000)
magenta = (255, 000, 255)
yellow  = (000, 255, 255)

# Setup Blob Tracker 1 with Parameters ----------------------------------------
# These parameters have to be tweaked, depending on the shape of the magnets...
params1 = cv2.SimpleBlobDetector_Params()
# Change Thresholds (10-100 is pretty perfect) (150 for a bit of noise)
params1.minThreshold =  10
params1.maxThreshold = 110
# Filter by Area (calibrate to image size scale ...) (20-160)
params1.filterByArea = True
params1.minArea = 150
params1.maxArea = 500
# Filter by Circularity (0.15 - 0.99)
params1.filterByCircularity = False
params1.minCircularity = 0.15
params1.maxCircularity = 0.99
# Filter by Convexity (0.8)
params1.filterByConvexity = False
params1.minConvexity = 0.8
#Filter by Inertia ("how elliptical are the blobs?") (0.05 - 1.0)
params1.filterByInertia = False
params1.minInertiaRatio = 0.05
params1.maxInertiaRatio = 1
detector1 = cv2.SimpleBlobDetector_create(params1) #

# Setup Blob Tracker 2 with Parameters ----------------------------------------
# Parameters, tweaked for the LEFT registration markers (perfect circles)
```

```
66 params2 = cv2.SimpleBlobDetector_Params()
67 # Change Thresholds (10-50 seems good, not too precise)
68 params2.minThreshold = 10
69 params2.maxThreshold = 80
70 # Filter by Area (~1150 for this marker)
71 params2.filterByArea = True
72 params2.minArea = 800
73 params2.maxArea = 1700
74 # Filter by Circularity (circle = 1)
75 params2.filterByCircularity = True
76 params2.minCircularity = 0.500
77 params2.maxCircularity = 1.010
78 # Filter by Convexity (area blob / area convex hull)
79 params2.filterByConvexity = True
80 params2.minConvexity = 0.85
81 params2.maxConvexity = 1.01
82 #Filter by Inertia (circle = 1, line = 0)
83 params2.filterByInertia = True
84 params2.minInertiaRatio = 0.70
85 params2.maxInertiaRatio = 1.01
86 detector2 = cv2.SimpleBlobDetector_create(params2)
87
88 # Setup Blob Tracker 3 with Parameters ----------------------------------------
89 # Parameters, tweaked for the RIGHT registration markers (perfect rectangles)
90 params3 = cv2.SimpleBlobDetector_Params()
91 # Change Thresholds (10-50 seems good, not too precise)
92 params3.minThreshold = 10
93 params3.maxThreshold = 80
94 # Filter by Area (~1600 for these markers)
95 params3.filterByArea = True
96 params3.minArea = 1250
97 params3.maxArea = 2000
98 # Filter by Circularity (circle = 1)
99 params3.filterByCircularity = True
100 params3.minCircularity = 0.000
101 params3.maxCircularity = 0.550
102 # Filter by Convexity (area blob / area convex hull)
103 params3.filterByConvexity = True
104 params3.minConvexity = 0.85
105 params3.maxConvexity = 1.01
106 #Filter by Inertia (circle = 1, line = 0)
107 params3.filterByInertia = True
108 params3.minInertiaRatio = 0.0000
109 params3.maxInertiaRatio = 0.2000
110 detector3 = cv2.SimpleBlobDetector_create(params3)
111
112 # Setup Blob Tracker 4 with Parameters ----------------------------------------
113 # Parameters, tweaked for the WORKSPACE registration markers (crosses)
114 params4 = cv2.SimpleBlobDetector_Params()
115 # Change Thresholds (10-50 seems good, not too precise)
116 params4.minThreshold = 10
117 params4.maxThreshold = 90
118 # Filter by Area (~1600 for these markers)
119 params4.filterByArea = True
120 params4.minArea = 1900
121 params4.maxArea = 3100
122 # Filter by Circularity (circle = 1)
123 params4.filterByCircularity = True
124 params4.minCircularity = 0.00
125 params4.maxCircularity = 0.45
126 # Filter by Convexity (area blob / area convex hull)
127 params4.filterByConvexity = True
128 params4.minConvexity = 0.00
129 params4.maxConvexity = 0.78
130 #Filter by Inertia (circle = 1, line = 0)
131 params4.filterByInertia = True
132 params4.minInertiaRatio = 0.65
133 params4.maxInertiaRatio = 1.01
134 detector4= cv2.SimpleBlobDetector_create(params4)
135
136
137
```

```
138 # FUNCION definitions ============================================================
139 def get_coordinates(keypoints):
140     # INPUT: raw keypoints from the detector (special format)
141     # ----------------------------------------------------------------------------
142     # The keypoint array contains different information about the blobs. This
143     # function extracts only the coordinates and stores them in more practical
144     # vector "coords" OpenCV and Numpy interpret y coordinates exactly inverted
145     # ----------------------------------------------------------------------------
146     frame_height = 1024 # <-------------------------------- enter FRAME HEIGHT
147     keypoint_coordinates = []
148     i_inverted_y = (0, 0)
149     for i in keypoints:
150         # inverts the y axis (adjust with image height!)
151         i_inverted_y = (i.pt[0], -i.pt[1] + frame_height)
152         # append i_inverted to final coordinate list
153         keypoint_coordinates.append( (i.pt[0], i.pt[1]) )
154     keypoint_coordinates.sort() # sorts the tuples by x. This defines
155     return keypoint_coordinates # OUTPUT: only coordinates as tuple list:
156 
157 
158 def get_distances(keypoint_coordinates):
159     # INPUT: extracted coordinates from keypoints (tuple list)
160     # ----------------------------------------------------------------------------
161     # This function creates an N times N matrix with all the distances between
162     # every possible pair of points. This is mainly done to save on computing
163     # resources, since they are used multiple times
164     # ----------------------------------------------------------------------------
165     N = len(keypoint_coordinates)
166     dist_matrix = np.zeros((N, N))
167     for i in range(0, N):
168         for k in range(0, N):
169             # (if starting @i, would only calculate the upper half of Matrix)
170             x1 = keypoint_coordinates[i][0]
171             x2 = keypoint_coordinates[k][0]
172             y1 = keypoint_coordinates[i][1]
173             y2 = keypoint_coordinates[k][1]
174             summa = (x1 - x2)**2 + (y1 - y2)**2
175             dist_matrix[i, k] = np.sqrt(summa)
176     return dist_matrix # OUTPUT: Matrix with all distances, global point ORDER!
177 
178 
179 def get_angle(p1, p2, p3, coords, dist_matrix):
180     # INPUT: three "consecutive" points
181     # ----------------------------------------------------------------------------
182     # This function calculates the outer angle between 3 consecutive points
183     # (0 if inline). This is done by calculating two vectors and applying the
184     # angle formula. To distinguish between + and -, the sign from their cross
185     # product is used. This is needed, because the catheter can go to the left
186     #  or "to the right"
187     # ----------------------------------------------------------------------------
188     # forming vector p1 -> p2
189     a = [ coords[p2][0] - coords[p1][0], coords[p2][1] - coords[p1][1] ]
190     # forming vector p2 -> p3
191     b = [ coords[p3][0] - coords[p2][0], coords[p3][1] - coords[p2][1] ]
192     # angle-formula for two vectors
193     val = np.dot(a, b) / (dist_matrix[p1, p2] * dist_matrix[p2, p3])
194     signum = np.sign(np.cross(a, b)) # cross product
195     angle123 = signum *np.arccos(val) *180 /np.pi # convert to degrees
196     return angle123 # OUTPUT: angle in degrees
197 
198 
199 def my_distribution(x, avg_length):
200     # INPUT: distance to the possible next point and average distance
201     # ----------------------------------------------------------------------------
202     # This function calculates, how likely it is, that the next closest point
203     # to an existing, unfinished catheter chain is part of it. Double the
204     # average distance is prop. as unlikely as half the average distance.
205     # ----------------------------------------------------------------------------
206     p_closer = 1.5 # defines the exponent of the weight function <------- param
207     p_further = 5.0 # defines the exponent of the weight function <-------param
208     x = 1.0*abs(x)
209     if ( (x > 0) & (x < avg_length) ): # when point is closer than average dist
```

```
210         return 1 + (abs(x-avg_length)/x)**p_closer
211     elif (x >= avg_length): # when point is further than average dist.
212         return 1 + ((x-avg_length)/avg_length)**p_further
213     else: # to avoid errors, return 1 when x = avg_length
214         return 1 # OUTPUT: probability of being "next point" and not just noise
215
216
217 def rotate_vec(vec, alpha):
218     # INPUT: 2D vector and rotation angle in Rad (+ = counterclockwise)
219     Rot = [ [np.cos(alpha), -np.sin(alpha)],
220             [np.sin(alpha),  np.cos(alpha)] ]
221     vec = [ np.dot(Rot[0], vec), np.dot(Rot[1], vec) ]
222     return vec # OUTPUT: rotated vector
223
224
225 def get_chain_score(chain, coords, dist_matrix, wL, wA, wU):
226     # INPUT: one of the initial chains plus weights L, A, U
227     # ----------------------------------------------------------------------------
228     # This function is used to initially score all the short chains, formed for
229     # every single point. They are only roughly judged by their average lenght
230     # and average angle change (= how many kinks the chain has). The goal is to
231     # find ONE point, that's highly likely to lie on the catheter. This is
232     # sufficient. The weights define how important each criterium is.
233     # ----------------------------------------------------------------------------
234     n_steps = len(chain)
235     if n_steps < 4:
236         # length-check: consecutive angle difference only possiblev for 4+
237         print("chain too short for Angle calc")
238         return 999, 999, 999, 999 # should never even be called
239     length_score = 0
240     for k in range(0, n_steps-1):
241         # adds up and averages all distances in the chain
242         length_score += dist_matrix[chain[k], chain[k+1]]
243     length_score = (1/(n_steps-1)) *length_score
244     angle_score = 0
245     for k in range(0, n_steps-3):
246         # adds up and averages all the consecutive angle differences
247         p1 = chain[k]
248         p2 = chain[k+1]
249         p3 = chain[k+2]
250         p4 = chain[k+3]
251         angle_score += (abs( get_angle(p1, p2, p3, coords, dist_matrix)
252             - get_angle(p2, p3, p4, coords, dist_matrix) ))
253     angle_score = (1/(n_steps-3)) *angle_score
254     uniformity_score = 1
255
256     total_score = ((1/wL)*length_score
257         + (1/wA)*angle_score
258         + (1/wU)*uniformity_score) /3 # a good chain should be ~1.0
259     # OUTPUT: criterium for quickly judging first chains
260     return total_score, length_score, angle_score, uniformity_score
261
262
263 def score_for_calibration(best_point, coords, dist_matrix, n_steps, wL, wA, wU)
264     # INPUT: point that's likeliest on cath
265     # ----------------------------------------------------------------------------
266     # this function is only convenience for recalibrating the length-weight
267     # (since image scaling isn't always the same)
268     # (as a workaround, it quickly builds the chain again)
269     # ----------------------------------------------------------------------------
270     N = len(coords)
271     dist_masked = ma.masked_array(dist_matrix, mask = np.identity(N, int))
272     chain = []
273     P = best_point # from this point (index)
274     Pnext = 0 # the next closest is searched (index)
275     for repeat in range (0, n_steps):
276         chain.append(P) # log the point
277         Pnext = np.argmin(dist_masked[P]) # find the next closest point (index)
278         dist_masked[:, P] = ma.masked_array( dist_masked[:, P],
279             mask = np.ones(N) ) # mask Column of used point
280         # This loop could be optimized, by applying the maks before searching..
281         P = Pnext
```

```
282     # OUTPUT: output score of this one (best) chain
283     return get_chain_score(chain, coords, dist_matrix, wL, wA, wU)
284
285
286 def score_all_points(coords, dist_matrix, n_steps, wL, wA, wU):
287     # INPUT: coordinates and distances in global point order
288     # ------------------------------------------------------------------------
289     # This function handles the scoring of all keypoints. This is done as a
290     # first step to determine ONE starting point, which is certain to lie on th
291     # catheter. This method is pretty reliable, even though the chains are
292     # judged only by length + angle changes and they are rather short(~5 steps)
293     # ------------------------------------------------------------------------
294     N = len(coords)
295     if n_steps > N: # should not be possible to enter here...
296         print("Requested Chain Lenght too long")
297         return
298     if n_steps < 4: # should not be possible to enter here...
299         print("Requested Chain Length too short")
300         return
301
302     # to avoid finding itself as closest point
303     dist_masked = ma.masked_array(dist_matrix, mask = np.identity(N, int))
304     scores = [] # all scores are stored here (score, point index)-List
305     for i in range(0, N):
306         # this loop cycles through all points (1->N) and builds a chain of leng
307         # n_steps for each one. Finally the finished chain is scored.
308          # restore "fresh" matrix every time
309         dist_masked = ma.masked_array(dist_matrix, mask = np.identity(N, int))
310         chain = []
311         P = i # from this point (index)
312         Pnext = 0 # the next closest is searched (index)
313         for repeat in range (0, n_steps): # this loop actually builds chain
314             chain.append(P) # log the point
315             Pnext = np.argmin(dist_masked[P]) # find the next closest point
316             dist_masked[:, P] = ma.masked_array( dist_masked[:, P],
317                 mask = np.ones(N) ) # mask Column of used point
318             # This loop could be optimized, by applying the maks before search.
319             P = Pnext
320         scores.append( (get_chain_score(chain,
321             coords,
322             dist_matrix,
323             wL, wA, wU)[0],
324             i) ) # chain score is calculated
325      # OUTPUT: List of all the points scores in "global point order"
326     return scores
327
328
329 def expand_catheter(coords, dist_matrix, catheter, wL, wA, wU, recursion_depth)
330     #INPUT: current, unfinished catheter
331     # ------------------------------------------------------------------------
332     # This function is called recursively, to expand the unfinished
333     # catheter-chain by the next best point. It does so, by looking at both
334     # ends, finding the two closest points at each location and then taking
335     # the one, which has the best "score" for angle and length diff
336     # (similar judgement to "chain_score Function")
337     # ------------------------------------------------------------------------
338     N = len(coords)
339     spread = 2 # number of close points tested each run (both at head and tail)
340     # to avoid finding itself as closest P.
341     dist_masked = ma.masked_array(dist_matrix, mask = np.identity(N, int))
342     # mask of all points, that are already in the catheter
343     ma.getmask(dist_masked)[:, catheter] = 1
344     head = catheter[0] # first element is the head
345     tail = catheter[-1] # last element is the tail
346     next_head = []
347     next_tail = []
348     possible_scores = []
349     add_score = 0
350
351     wL_expander = 0.5 # adjust prioritizing length or angle score
352     wA_expander = 0.4
353
```

```
354     for i in range(0, spread):
355         # get index of closest n points to head.
356         # Same chain algorithm as in score_all_points
357         closest = np.argmin(dist_masked[head])
358         next_head.append(closest)
359         ma.getmask(dist_masked)[head, closest] = 1
360         # mask off the point that has just been found, but only for searches
361         # coming from this points row! (single cell mask). By doing it this way
362         # the matrix can be reused for searches, coming from othe points
363     for i in range(0, spread):
364         # get index of closest n points to tail.
365         # Same chain algorithm as in score_all_points
366         closest = np.argmin(dist_masked[tail])
367         next_tail.append(closest)
368         ma.getmask(dist_masked)[tail, closest] = 1
369         # mask off the point that has just been found, but only for searches
370         # coming from this row! (single cell mask). By doing it this way,
371         # the matrix can be reused for searches, coming from othe points
372 
373     for P in next_head:
374         # calculates and logs the score for all possible new chains
375         # with the newly found points
376         p1 = catheter[0]
377         p2 = catheter[1]
378         p3 = catheter[2]
379         anglediff = get_angle(P, p1, p2, coords, dist_matrix)
380             - get_angle(p1, p2, p3, coords, dist_matrix)
381         # here a score is calculated, for every single possible new point.
382         # This score judges, how much the new point is angled away from the
383         # head of the catheter and much the distance to the new point
384         # deviates from the average distance between points, that already
385         # belong to the catheter
386         add_score = wL_expander*my_distribution(dist_matrix[P, p1], wL)
387             + wA_expander*((1/wA)*abs(anglediff))**1.5 # <--
388         possible_scores.append( (add_score, True, P) )
389         # True as second argument is used to identify as "head"
390     for P in next_tail:
391         p1 = catheter[-3]
392         p2 = catheter[-2]
393         p3 = catheter[-1]
394         anglediff = get_angle(p1, p2, p3, coords, dist_matrix)
395             - get_angle(p2, p3, P, coords, dist_matrix)
396         add_score = wL_expander*my_distribution(dist_matrix[p3, P], wL)
397             + wA_expander*((1/wA)*abs(anglediff))**1.5 # <--
398         # same score and criteria, but calculated w.r.t tail...
399         possible_scores.append( (add_score, False, P) )
400         # False as second argument is used to identify as "tail"
401 
402     possible_scores.sort()
403     if (possible_scores[0][0] > 50):
404         # breaks if the best score becomes too large (~5)
405         # print("killed@: ", possible_scores[0][0])
406         return catheter # returns the FINISHED catheter
407     if (len(catheter) >= N-1): # breaks if all points have been exhausted
408         return catheter # returns the FINISHED catheter
409     if (recursion_depth >= 30):
410         # recursion failsafe, this is basically used, if the catheter has more
411         # keypoints, than expected, or something went wrong, and the recursion
412         # would not stop otherwise. Should ideally not be entered...
413         print("Error: recursion went too far")
414         return catheter # returns the CORRUPTED catheter
415 
416     if (possible_scores[0][1]): # update Catheter, append point at head (True)
417         catheter.insert(0, possible_scores[0][2])
418     else: # update Catheter, append new point at tail (False)
419         catheter.append(possible_scores[0][2])
420     recursion_depth += 1
421     # using recursion, this function is called again, but the newly updated
422     # version of the catheter is passed on. With every recursion step, the
423     # catheter will grow by one point. When the last function finds no new
424     # point, the final catheter is handed back (through all return statements)
425     return expand_catheter(coords, dist_matrix, catheter,
```

```
426         wL, wA, wU, recursion_depth) # OUTPUT: finished Catheter
427
428
429 def build_catheter(coords, dist_matrix, n_steps, Pstart, wL, wA, wU):
430     # INPUT: Starting point
431     # ------------------------------------------------------------------------
432     # This function is mostly just convenience to handle the recursive
433     # "expand_catheter" routine. It also builds the first four points of cath-
434     # chain, since they are needed, in order for expand_catheter to work...
435     # (The first four points, are the same as constructed in "score_all_points"
436     # hence they should be safe to add to the catheter)
437     # ------------------------------------------------------------------------
438     recursion_depth = 0
439     N = len(coords)
440     dist_masked = ma.masked_array(dist_matrix, mask = np.identity(N, int))
441     catheter = []
442     P = Pstart
443     Pnext = 0
444     for repeat in range(0, n_steps): # build the first n_step Elements
445         catheter.append(P) # log the point
446         Pnext = np.argmin(dist_masked[P]) # find the next closest point (index)
447         dist_masked[:, P] = ma.masked_array( dist_masked[:, P],
448             mask = np.ones(N) ) # mask Column of used point
449         # This loop could be optimized, by applying the maks before search.
450         P = Pnext
451
452     # after the first n_step elements have been built, the recursive tool for
453     # building the catheter is started by this function call.
454     catheter = expand_catheter(coords, dist_matrix, catheter,
455         wL, wA, wU, recursion_depth) # this yields final catheter
456     #OUTPUT: chain list of the whole catheter, and only the catheter (FINISHED)
457     return catheter
458
459
460 def error_stamper(error_image, text, height):
461     # INPUT: Image, where no catheter could be found, and text message string
462     # ------------------------------------------------------------------------
463     # This function will just add an error message to an input image
464     # ------------------------------------------------------------------------
465     font = cv2.FONT_HERSHEY_SIMPLEX
466     font_scale = 1
467     thickness = 6
468     stamped_image = cv2.putText(error_image, text, (200, height),
469         font, font_scale, red, thickness)
470     return stamped_image # OUTPUT: image with text stamped onto it
471
472
473 def draw_tip(frame, tipsection_x, tipsection_y):
474     # INPUT: frame and info about on which side the tip of the catheter lies
475     # ------------------------------------------------------------------------
476     # this function is just convenience and colors the tip of catheter green
477     # ------------------------------------------------------------------------
478     frame= cv2.circle(frame, (int(tipsection_x[7]), int(tipsection_y[7])),
479         12, green, 2)
480     for x, y in zip(tipsection_x, tipsection_y):
481         frame= cv2.circle(frame, (int(x), int(y)), 3, green, 2)
482     # OUTPUT: frame with green circles at tip
483     return frame
484
485
486 def draw_angle_overlay(frame, tipsection_x, tipsection_y, uvec):
487     # INPUT: the tip of the catheter
488     # ------------------------------------------------------------------------
489     # this function is just convenience and draws the angle-information-overlay
490     # ------------------------------------------------------------------------
491     slope = np.polyfit(tipsection_x, tipsection_y, 1)[0]
492     magnitude = np.sqrt(slope**2 + 1)
493     cathvec = [1/magnitude, slope/magnitude]
494     if (np.dot(uvec, cathvec) < 0):
495         # this point-mirrors the vec, if its + angle to uvec is > 90
496         cathvec[0] = -cathvec[0]
497         cathvec[1] = -cathvec[1]
```

```
498     # distinguish between going to the left and to the right
499     signum = np.sign(np.cross(uvec, cathvec))
500     # angle w.r.t. path line, in Rad
501     phi = -signum * np.arccos(np.dot(uvec, cathvec))
502 
503     ind_L = 250 # define length of indicator
504     p1 = ( int(tipsection_x[2]), int(tipsection_y[2]) )
505     p2 = ( int(p1[0] + cathvec[0]*ind_L), int(p1[1] + cathvec[1]*ind_L) )
506     cv2.line(frame, p1, p2, green, 5) # angle shadow
507     cv2.line(frame, p1, p2, red, 2) # angle inner
508     text = " angle: %s deg" %round(phi/2/np.pi*360, 1)
509     font = cv2.FONT_HERSHEY_SIMPLEX
510     font_scale = 1
511     cv2.putText(frame, text , p2, font, font_scale, black, 5)
512     cv2.putText(frame, text , p2, font, font_scale, white, 2)
513     return frame, phi # OUTPUT: frame with overlays and the angle in deg
514 
515 
516 def draw_curvature_overlay(frame, tipsection, tipsection_x, tipsection_y, uvec,
517     tipsize, coords, dist_matrix, phi):
518     # ------------------------------------------------------------------------
519     # this function is just convenience and draw the curvature-overlay
520     # ------------------------------------------------------------------------
521     tot_tip_angle = 0
522     for i in range(0, tipsize-2):
523         tot_tip_angle += abs(get_angle(tipsection[i], tipsection[i+1],
524             tipsection[i+2], coords, dist_matrix))**3
525     # avg angle per segment in Rad
526     avg_tip_angle = np.cbrt(tot_tip_angle/360*2*np.pi)/tipsize/5
527     length = 22
528     signum = np.sign(phi)
529     phi = abs(phi)
530 
531     tvec = rotate_vec(uvec, -signum*avg_tip_angle-signum*phi)
532     p1 = (int(tipsection_x[2]), int(tipsection_y[2]))
533     p2 = ( int(p1[0] + tvec[0]*length), int(p1[1] + tvec[1]*length) )
534     cv2.line(frame, p1, p2, green, 5) # curvature shadow
535     cv2.line(frame, p1, p2, red, 2) # curvature inner
536     for i in range(0, 7): # actually 8 times but once is done up there
537         tvec = rotate_vec(tvec, -signum*avg_tip_angle)
538         p1 = p2
539         p2 = ( int(p1[0] + tvec[0]*length), int(p1[1] + tvec[1]*length) )
540         cv2.line(frame, p1, p2, green, 5)
541         cv2.line(frame, p1, p2, red, 2)
542     text = " curv.avg: %s deg" %round(signum*avg_tip_angle/2/np.pi*360, 1)
543     font = cv2.FONT_HERSHEY_SIMPLEX
544     font_scale = 1
545     cv2.putText(frame, text , p2, font, font_scale, black, 6)
546     cv2.putText(frame, text , p2, font, font_scale, white, 2)
547     p1 = (int(tipsection_x[2]), int(tipsection_y[2]))
548     cv2.line(frame, p1, (int(p1[0]+uvec[0]*150), int(p1[1]+uvec[1]*150)),
549         green, 5)
550     cv2.line(frame, p1, (int(p1[0]+uvec[0]*150), int(p1[1]+uvec[1]*150)),
551         red, 2)
552     # OUTPUT, overlayed frame and curvature in deg
553     return frame, signum*avg_tip_angle/2/np.pi*360
554 
555 
556 
557 
558 # CLASS definitions ===========================================================
559 class Blob_Tracker_With_Counter:
560     def __init__(self):
561         # --------------------------------------------------------------------
562         # constructor of this class
563         # --------------------------------------------------------------------
564         print("initialize tracker ... ... ...")
565         self.pubxray = rospy.Publisher("xray_tracking_overlay",
566             Image, queue_size=1)
567         self.pubinfo = rospy.Publisher("catheter_info",
568             cath_info, queue_size=1)
569         self.sub = rospy.Subscriber("emulated_xray",
```

```
570             Image, self.callback_func) #/xray/image_display
571
572         # initialize some parameters ----------------------------------------
573         # essentially define all global variables here... if you then want to
574         # change them, do it here, inside of class definition! (essentially all
575         # variables used over multiply tracking cycles have to be stored here!)
576         # ----------------------------------------------------------------------
577         self.cal_counter = 99999
578         self.reg_counter = 99999
579         self.n_steps = 8 # chain length tested for score_all_points <----------
580         self.alpha = 1.40 # contrast control (1.0 - 3.0) <---------------------
581         self.beta  = -60 # Brightness control (0 - 50) (130)
582         self.wL = 50 # roughly set initial weights (will self calibrate) <-----
583         self.wA = 8 # set angle weight, will NOT self calibrate
584         self.wU = 1  # (not implemented!)
585         self.path_uvec = False
586         self.path_orig = False
587         self.navion_orig = False
588         self.thereiscath = False
589         self.thereismodel = False
590         self.thereisnavion = False
591         self.m_per_pixel = 0.0001 # rough value for how many meters per pixel
592
593
594     def callback_func(self, image_msg_in):
595         # ----------------------------------------------------------------------
596         # when an image is received, this function is called
597         # the actual WORK is done in this function [raw --> image with overlay]
598         # messages from the subscribed topic are passed into this function
599         # ----------------------------------------------------------------------
600         self.reg_counter += 1
601         self.cal_counter += 1
602         # convert ROS-message (sensor_msgs/Image) to cv2 image
603         frame_in = bridge.imgmsg_to_cv2(image_msg_in, "bgr8")
604         frameAdjusted = cv2.convertScaleAbs(frame_in, alpha=self.alpha,
605             beta=self.beta) # adjust Contrast and Brightness
606         keypoints1 = detector1.detect(frameAdjusted)
607         style = cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
608         frame_with_keypoints1 = cv2.drawKeypoints(frameAdjusted,
609             keypoints1, np.array([]), blue, style)
610
611         # UPDATE REGISTRATION Markers every n Frames -------------------------
612         if (self.reg_counter >= 30): # <------------------------------ REG count
613             self.reg_counter = 0 # reset counter
614             # update model orientation
615             keypoints2, keypoints3 = self.do_model_registration(frameAdjusted)
616             if (not self.thereisnavion):
617                 # update workspace origin
618                 keypoints4 = self.do_workspace_registration(frameAdjusted)
619             frame_with_keypoints1 = cv2.drawKeypoints(frame_with_keypoints1,
620                 keypoints2, np.array([]), cyan, style)
621             frame_with_keypoints1 = cv2.drawKeypoints(frame_with_keypoints1,
622                 keypoints3, np.array([]), yellow, style)
623             if (not self.thereisnavion):
624                 frame_with_keypoints1 = cv2.drawKeypoints(frame_with_keypoints1
625                     keypoints4, np.array([]), magenta, style)
626             cv2.putText(frame_with_keypoints1, "REG", (530, 50),
627                 cv2.FONT_HERSHEY_SIMPLEX, 1, magenta, 6)
628
629         # update weights (calibration) every n Frames -----------------------
630         if (self.cal_counter >= 30): # <-------------------------- CALIB count
631             self.cal_counter = 0 # reset counter
632             self.do_recalibration(keypoints1)
633
634         # Draw PATH ---------------------------------------------------------
635         if (self.path_uvec and self.path_orig):
636             p1 = (self.path_orig[0], self.path_orig[1])
637             pathlength = 400
638             p0 = (int(p1[0] - self.path_uvec[0]*pathlength),
639                 int(p1[1] - self.path_uvec[1]*pathlength))
640             p2 = (int(p1[0] + self.path_uvec[0]*pathlength),
641                 int(p1[1] + self.path_uvec[1]*pathlength))
```

```
642                 cv2.circle(frame_with_keypoints1, p1, 10, cyan, 2) # shadow
643                 cv2.circle(frame_with_keypoints1, p1, 8, magenta, 2) # circle
644                 cv2.line(frame_with_keypoints1, p0, p2, cyan, 5) # hadow
645                 cv2.line(frame_with_keypoints1, p0, p2, magenta, 3)  # line
646                 cv2.putText(frame_with_keypoints1, " path", p2,
647                     cv2.FONT_HERSHEY_SIMPLEX, 1, cyan, 5)
648                 cv2.putText(frame_with_keypoints1, " path", p2,
649                     cv2.FONT_HERSHEY_SIMPLEX, 1, magenta, 2)
650                 # Continue
651             else:
652                 frame_with_keypoints1 = error_stamper(frame_with_keypoints1,
653                     "no path found (Marker error)", 150)
654                 # Continue <------------------------------------------------ NO REG
655 
656             # Draw NAVION origin ---------------------------------------------------
657             if (self.navion_orig):
658                 og = ( int(self.navion_orig[0]), int(self.navion_orig[1]) )
659                 cv2.line(frame_with_keypoints1,
660                     (og[0]-10, og[1]-10), (og[0]+10, og[1]+10), cyan, 5)
661                 cv2.line(frame_with_keypoints1,
662                     (og[0]-10, og[1]+10), (og[0]+10, og[1]-10), cyan, 5)
663                 cv2.circle(frame_with_keypoints1, og, 6, magenta, -1)
664                 cv2.circle(frame_with_keypoints1, og, 8, cyan, 2)
665                 cv2.putText(frame_with_keypoints1, " navion orig",
666                     og, cv2.FONT_HERSHEY_SIMPLEX, 1, cyan, 5)
667                 cv2.putText(frame_with_keypoints1, " navion orig",
668                     og, cv2.FONT_HERSHEY_SIMPLEX, 1, magenta, 2)
669             else:
670                 frame_with_keypoints1 = error_stamper(frame_with_keypoints1,
671                     "no navion origin found", 250)
672 
673             # Do CATHETER recognition and drawing ----------------------------------
674             # exit, if less than 5 keypoints or no path_uvec!
675             if (len(keypoints1) < self.n_steps or not self.path_uvec):
676                 if (not self.path_uvec):
677                     frame_out = error_stamper(frame_with_keypoints1,
678                         "no path_uvec defined!", 200)
679                 if (len(keypoints1) < self.n_steps):
680                     frame_out = error_stamper(frame_with_keypoints1,
681                         "no catheter found (keypoint error)", 100)
682                 self.thereiscath = False
683                 # BREAK <------------------------------------------- NO CATHETER
684             else: # when there are (8) or more keypoints, do cath recognition
685                 coords = get_coordinates(keypoints1)
686                 coords_x = [p[0] for p in coords]
687                 coords_y = [p[1] for p in coords]
688                 dist_matrix = get_distances(coords)
689                 scores = score_all_points(coords, dist_matrix,
690                     self.n_steps, self.wL, self.wA, self.wU)
691                 scores.sort()
692 
693                 if (scores[0][0] > 1.5):
694                 # when all chains are too bad, exit [0.7 - 5] possible
695                     frame_out = error_stamper(frame_with_keypoints1,
696                         "no catheter found (bad chains error)", 100)
697                     self.thereiscath = False
698                     # BREAK, No catheter found <---------------------- NO CATHETER
699                 else: # if there is a good chain, try BUILDING CATHETER
700                     Pstart = scores[0][1]
701                     catheter = build_catheter(coords, dist_matrix,
702                         self.n_steps, Pstart, self.wL, self.wA, self.wU)
703 
704                     frame_with_catheter = frame_with_keypoints1
705                     for p in catheter: # draw all keypoints in red
706                         int_coords_cath = ( (int(coords_x[p]), int(coords_y[p])) )
707                         frame_with_catheter = cv2.circle(frame_with_catheter,
708                             int_coords_cath, 6, red, 3) # <-- DONE
709 
710                     # find the tip of the catheter -----------------------------
711                     templist = []
712                     head = catheter[0]
713                     tail = catheter[-1]
```

```
714                 dotprod_head = np.dot( self.path_uvec,
715                     [coords_x[head], coords_y[head]] )
716                 dotprod_tail = np.dot( self.path_uvec,
717                     [coords_x[tail], coords_y[tail]] )
718                 templist.append( (dotprod_head, True ) ) # True for head
719                 templist.append( (dotprod_tail, False) ) # False for tail
720                 # tip point is the one which is furthest along path_uvec
721                 thetip = max(templist, key=lambda x:x[0])
722
723                 tipsection_x = [] # x-coordinates of tip-points
724                 tipsection_y = [] # y-coordinates of tip-points
725                 tipsection = [] # ordered point indices of tip-points
726                 tipsize = self.n_steps # <--------------------------- TIP SIZE
727                 # for curvature, angle and pos calculation only the tip is
728                 # needed, which is stored in "tipsection"
729                 if thetip[1]: #add from 0
730                     # this loop builds the tipsection from the "head end"
731                     for p in catheter[0:tipsize]:
732                         tipsection_x.append(coords_x[p])
733                         tipsection_y.append(coords_y[p])
734                         tipsection.append(p)
735                 else: # add from -1
736                     # this loop build the tipsection from the "tail end"
737                     for p in catheter[-1:-(tipsize+1):-1]:
738                         tipsection_x.append(coords_x[p])
739                         tipsection_y.append(coords_y[p])
740                         tipsection.append(p)
741
742                 frame_with_catheter = draw_tip(frame_with_catheter,
743                     tipsection_x, tipsection_y)
744                 frame_with_angle, phi = draw_angle_overlay(frame_with_catheter,
745                     tipsection_x, tipsection_y, self.path_uvec)
746                 frame_with_curvature, curvature = draw_curvature_overlay(
747                         frame_with_angle, tipsection, tipsection_x,
748                         tipsection_y, self.path_uvec, tipsize,
749                         coords, dist_matrix, phi)
750                 frame_out = frame_with_curvature
751                 self.thereiscath = True
752
753         # handle output publication ----------------------------------------
754         info_msg_out = cath_info()
755         info_msg_out.m_per_pixel = self.m_per_pixel
756         if (self.thereiscath):
757             info_msg_out.pos_x = tipsection_x[7]
758             info_msg_out.pos_y = tipsection_y[7]
759             info_msg_out.phi = phi
760             info_msg_out.avg_curv = curvature
761             info_msg_out.thereiscath = True
762         else:
763             info_msg_out.thereiscath = False
764
765         if (self.thereismodel):
766             info_msg_out.path_uvec_x = self.path_uvec[0]
767             info_msg_out.path_uvec_y = self.path_uvec[1]
768             info_msg_out.path_orig_x = self.path_orig[0]
769             info_msg_out.path_orig_y = self.path_orig[1]
770             info_msg_out.thereismodel = True
771         else:
772             info_msg_out.thereismodel = False
773
774         if (self.thereisnavion):
775             info_msg_out.navion_orig_x = self.navion_orig[0]
776             info_msg_out.navion_orig_y = self.navion_orig[1]
777             info_msg_out.thereisnavion = True
778         else:
779             info_msg_out.thereisnavion = False
780
781         image_msg_out = bridge.cv2_to_imgmsg(frame_out, "bgr8")
782         self.pubxray.publish(image_msg_out)
783         self.pubinfo.publish(info_msg_out)
784
785
```

```
786     def do_recalibration(self, keypoints1):
787         # ------------------------------------------------------------------------
788         # this calibrates wL, using the best scoring initial chain. As
789         # as workaround, the chain has to be built here as well...
790         # necessary, since image resolution could technically change, and
791         # therefore also change "unit lenght" in this code
792         # ------------------------------------------------------------------------
793         if (len(keypoints1) >= self.n_steps):
794         # abort calibration, if there are not enough points
795             coords = get_coordinates(keypoints1)
796             dist_matrix = get_distances(coords)
797             scores = score_all_points(coords, dist_matrix, self.n_steps,
798                 self.wL, self.wA, self.wU)
799             scores.sort()
800             best_point = scores[0][1]
801             if (scores[0][0] < 1.5): # to avoid calibration to bad chain
802                 new_wL = score_for_calibration(best_point, coords, dist_matrix,
803                     self.n_steps, self.wL, self.wA, self.wU)[1]
804                 self.wL = 0.7*self.wL + 0.3*new_wL
805                 # slowly adjust old wL to newly found one
806                 # wL is set to be the average distance, found in the lowest-
807                 # scoring chain of 5 points (this is equal to lenght score of
808                 # this chain) wA = score_for_calibration()[2] # wA can't be
809                 # calibrated sensibly!
810
811
812     def do_model_registration(self, frameAdjusted):
813         # ------------------------------------------------------------------------
814         # this function will occasionally re-registrate the spine path
815         # should only happen every few cycles / frames either. but still, needs
816         # to be updated every so often, to account for patient movement. ~1 sec
817         # ------------------------------------------------------------------------
818         keypoints2 = detector2.detect(frameAdjusted) # detects circle markers
819         keypoints3 = detector3.detect(frameAdjusted) # detects rectangle marker
820
821         if (len(keypoints2) >= 2 and len(keypoints3) >= 2):
822             keycoords2 = get_coordinates(keypoints2)
823             keycoords3 = get_coordinates(keypoints3)
824             # extract all x-coordinates of keypoints2
825             kx2 = [p[0] for p in keycoords2]
826             # extract all y-coordinates of keypoints2
827             ky2 = [p[1] for p in keycoords2]
828             kx3 = [p[0] for p in keycoords3]
829             ky3 = [p[1] for p in keycoords3]
830             a2, b2 = np.polyfit(kx2, ky2, 1)
831             a3, b3 = np.polyfit(kx3, ky3, 1)
832             a_midline = a2 # np.tan( 0.5*np.arctan(a2) + 0.5*np.arctan(a3) )
833             magnitude = np.sqrt(a_midline**2 + 1)
834             uvec = [1/magnitude, a_midline/magnitude]
835             LRvec = [kx3[0] - kx2[0], ky3[0] - ky2[0]]
836             if ( np.sign(np.cross(LRvec, uvec)) > 0 ):
837             # to distinguish between catheter going top-bottom or bottom-top
838                 uvec[0] = -uvec[0]
839                 uvec[1] = -uvec[1]
840
841             if (abs(uvec[1]) < 1/np.sqrt(2)):
842             # define origin (gives it a bit more robustnes against "poles")
843                 yLeft = a2*1024/2 + b2
844                 yRight = a3*1024/2 + b3
845                 orig = [int(1024/2), int(0.5*yLeft + 0.5*yRight)]
846             else:
847                 xLeft = (1024/2 - b2)/a2
848                 xRight = (1024/2 - b3)/a3
849                 orig = [int(0.5*xLeft + 0.5*xRight), int(1024/2)]
850             self.path_orig = orig
851             self.path_uvec = uvec
852             self.thereismodel = True
853             return keypoints2, keypoints3
854
855         else: # No markers and path have been detected!
856             # this ensures that another registration attempt
857             # is made as soon as possible
```

```
858             self.reg_counter = 9999
859             self.path_orig = False
860             self.path_uvec = False
861             self.thereismodel = False
862             return keypoints2, keypoints3
863 
864 
865     def do_workspace_registration(self, frameAdjusted):
866         keypoints4 = detector4.detect(frameAdjusted) # detect cross markers
867 
868         if (len(keypoints4) == 4):
869             keycoords4 = get_coordinates(keypoints4)
870             kx4 = [p[0] for p in keycoords4] # extract all x-coordinates
871             ky4 = [p[1] for p in keycoords4] # extract all y-coordinates
872             avg_x = sum(kx4)/4
873             avg_y = sum(ky4)/4
874             self.navion_orig = (avg_x, avg_y)
875             self.thereisnavion = True
876 
877             a = avg_x - keycoords4[0][0]
878             b = avg_y - keycoords4[0][1]
879             comparedist = np.sqrt(a**2 + b**2)
880             self.m_per_pixel = np.sqrt(32) /100 /comparedist
881 
882             return keypoints4
883         else: # no markers found
884             # this ensures that another registration attempt
885             # is made as soon as possible
886             self.reg_counter = 9999
887             self.navion_orig = False
888             self.thereisnavion = False
889             return keypoints4
890 
891 
892 
893 
894 # script protection ===========================================================
895 if (__name__ == "__main__"):
896     try:
897         # do work ---------------------------------------------------------------
898         rospy.init_node("blob_tracker_main")
899         Blob_Tracker_With_Counter()
900         rospy.spin()
901     except rospy.ROSInterruptException:
902         pass
```

A.2 Control-Node source-code

```
#!/usr/bin/env python
# the actual service, doing the blob tracking

from os import posix_fadvise
import numpy as np
from numpy.lib.utils import info
import numpy.ma as ma
import matplotlib.pyplot as plt
import scipy
import scipy.ndimage
import random
import time
import cv_bridge
import threading

import rospy
import sys
import cv2
from cv_bridge import CvBridge
from sensor_msgs.msg import Image
from mag_msgs.msg import FieldStamped
from geometry_msgs.msg import Point, Vector3

from epi_tracker.msg import cath_info
from std_msgs.msg import Float32

# for max magnetic field
from mag_manip.mag_manip import *
import rospkg
import os.path



# ============================================================================
# CODE main body ############################################################
# ============================================================================



# CONVENIENCE definitions ====================================================
bridge = CvBridge()



# FUNCTION definitions =======================================================
def rotate_vec(vec, alpha):
    # INPUT: 2D vector and rotation angle in Rad (+ = counterclockwise)
    alpha = alpha /360 *2 *np.pi
    Rot = [ [np.cos(alpha), -np.sin(alpha)],
            [np.sin(alpha),  np.cos(alpha)] ]
    vec = [ np.dot(Rot[0], vec), np.dot(Rot[1], vec) ]
    return vec # OUTPUT: rotated vector



# CLASS definition ===========================================================
class Controller_Main_Node:
    def __init__(self):
        # ------------------------------------------------------------------
        # constructor of this class
        # ------------------------------------------------------------------
        print("initialize Controller Node ... ... ...")
        self.sub = rospy.Subscriber("catheter_info",
            cath_info, self.callback_func)

        # initialize some parameters ---------------------------------------
        # essentially define all global variables here... if you then want to
        # change them, do it here, inside of class definition! (essentially all
        # variables used over multiply tracking cycles have to be stored here!)
```

```
70          # ------------------------------------------------------------------
71          self.timestamp = rospy.Time.now().secs
72              + rospy.Time.now().nsecs/1000000000 # sets an initial ROS-timestamp
73          self.delay = 1.50 # time delay between two control-steps (in seconds)
74
75          self.prev_cath_x = False # parameters to store previous catheter info
76          self.prev_cath_y = False
77          self.prev_cath_phi = False
78          self.prev_cath_avg_curv = False
79
80          # a couple parameters to keep track of passed control steps
81          self.is_evading = False
82          self.ev_depth = 0
83          # minus will mean corretion to the navion -y axis.
84          # -ev_width will add +errdist
85          self.ev_width = 0
86          self.consecutive_fwd = 0
87          self.allowed_err = 1
88
89
90      def callback_func(self, cath_info):
91          # ------------------------------------------------------------------
92          # when a cath_info message is received, this function is called.
93          # messages from the subscribed topic are passed on into this function
94          # ------------------------------------------------------------------
95          thereisC = cath_info.thereiscath
96          thereisN = cath_info.thereisnavion
97          thereisM = cath_info.thereismodel
98          isbentstuck = False
99          isnotmoving = False
100         delta = 9999
101
102         if ((rospy.Time.now().secs + rospy.Time.now().nsecs/1000000000
103             - self.timestamp) > self.delay):
104         # Checks, if enough time has passed since the last control step.
105         # Set self.delay to zero to circumvent this condition
106         # ------------------------------------------------------------------
107             templist = []
108             # templist contains names of all running Threads
109             for e in threading.enumerate(): templist.append(e.name)
110
111             if ("Threaded-Func" not in templist):
112             # Enters only, if currently no other control Thread is running!
113             # ------------------------------------------------------------
114
115                 if (thereisC and thereisN and thereisM):
116                 # this condition will only allow to continue, if all three
117                 # Elements are present (Catheter, Naion, Model). If this is not
118                 # the case, all prev-values are reset and no control is done
119                 # ------------------------------------------------------
120
121                     # ------------------------------------------------
122                     # The catheter is examined for bending and not moving.
123                     # ------------------------------------------------
124                     if (cath_info.avg_curv >= 9.0): # <------ check CURVE STUCK
125                         # catheter stuck because curved to tightly!
126                         isbentstuck = True
127                     else:
128                         isbentstuck = False
129
130                     if (self.prev_cath_x and self.prev_cath_x):
131                         # only go here if a previous positional value is stored
132                         deltax = cath_info.pos_x - self.prev_cath_x
133                         deltay = cath_info.pos_y - self.prev_cath_y
134                         # calculate distance moved in one step
135                         delta = np.sqrt(deltax**2 + deltay**2)
136                         rospy.loginfo("   ")
137                         rospy.loginfo("----- new ctrl step -----")
138                         rospy.loginfo(f"DELTA WAS: {delta}
139                             (req: {0.001 /cath_info.m_per_pixel})")
140                         if (delta < 0.001 /cath_info.m_per_pixel):
141                             # catheter is not moving much or at all!
```

```
142                                 isnotmoving = True
143                             else:
144                                 isnotmoving = False
145                         else:
146                             isnotmoving = False
147                         rospy.loginfo(f"-> isbent: {isbentstuck}
148                             | notmoving: {isnotmoving}")
149                         rospy.loginfo(f"AE = {self.allowed_err}")
150 
151                         # ------------------------------------------------------
152                         # All the decision Logic for CONTROL is one here!
153                         # ------------------------------------------------------
154                         if (self.is_evading): # go here if EVADING-YES
155 
156                             if (isbentstuck or isnotmoving):
157                                 # EVADING-YES + ERROR-YES: The catheter was backing
158                                 # up but was not successful. Back up further
159                                 # ------------------------------------------------
160                                 self.ev_depth += 1
161                                 self.is_evading = True
162                                 rospy.loginfo("backing up further")
163                                 self.DO_WORK_HELPER(cath_info, -1, self.ev_width)
164 
165                             else:
166                                 # EVADING-YES + ERROR-NO: The catheter was backing
167                                 # up and was successful. Go to the side and forward
168                                 # ------------------------------------------------
169                                 self.ev_width -= 1
170                                 self.is_evading = False
171                                 self.allowed_err = 0
172                                 # self.ev_depth has to stay!
173                                 rospy.loginfo("backing up Sucess!")
174                                 self.DO_WORK_HELPER(cath_info, +1, self.ev_width)
175 
176                         else: # go here if EVADING-NO
177                             # go here if EVADING-NO + ERROR-YES
178                             if ( (isbentstuck or isnotmoving)
179                                 and self.allowed_err < 1):
180 
181                                 if (self.ev_depth > 0):
182                                     # EVADING-NO + ERROR-YES + EV_DEPTH > 0:
183                                     # a previous evasion did not go back far enough
184                                     # Same obstacle was hit again. Go back further,
185                                     # than previous evasion (-1) and try again
186                                     # ------------------------------------------
187                                     rospy.loginfo(f"err! previous evasion failed.
188                                         depth: {self.ev_depth} and width:
189                                         {self.ev_width}")
190                                     self.allowed_err = 0
191                                     self.ev_depth += 2
192                                     self.is_evading = False
193                                     self.consecutive_fwd = 0
194                                     self.ev_width -= 1
195                                     self.DO_WORK_HELPER(cath_info,
196                                         -self.ev_depth, self.ev_width)
197 
198                                 else:
199                                     # EVADING-NO + ERROR-YES + EV_DEPTH = 0:
200                                     # after a long success run a new evasion starts
201                                     # resect success run as well...
202                                     # ------------------------------------------
203                                     self.ev_depth += 1
204                                     self.is_evading = True
205                                     self.consecutive_fwd = 0
206                                     rospy.loginfo(f"err! starting new evasion.
207                                         bent: {isbentstuck},
208                                         notmoving: {isnotmoving}")
209                                     self.DO_WORK_HELPER(cath_info, -2,
210                                         self.ev_width)
211 
212                             else:
213                                 # EVADING-NO + ERROR-NO: this is NORMAL fwd movem.
```

```
214                             # add one successful step and go further
215                             # ------------------------------------------------
216                             self.allowed_err = 1
217                             if (isbentstuck or isnotmoving):
218                                 print(f"taking one off because bent:
219                                     {isbentstuck} | notmoving: {isnotmoving}")
220                                 self.allowed_err -= 1
221                             self.consecutive_fwd += 1
222                             if (self.consecutive_fwd > self.ev_depth
223                                 and self.ev_depth > 0):
224                                 # afer a long successful fwd-run, all evasion
225                                 # methods are set back to 0 ...
226                                 self.ev_depth = 0
227                                 self.ev_width = 0
228                                 self.is_evading = False
229                             rospy.loginfo(f"good! going forward. Sucess:
230                                 {self.consecutive_fwd}")
231                             self.DO_WORK_HELPER(cath_info, +1, self.ev_width)
232
233                 else:
234                 # goes here, if any element could not be detected in the image
235                 # ------------------------------------------------------------
236                     print("cannot do control, something is missing!")
237                     self.prev_cath_x = False # UPDATE PREV values before exit
238                     self.prev_cath_y = False
239                     self.prev_cath_phi = False
240                     self.prev_cath_avg_curv = False
241
242
243     def DO_WORK_HELPER(self, cath_info, n_steps, ev_width):
244         # ------------------------------------------------------------------
245         # This is only convenience function since all cases do roughly the same
246         # ------------------------------------------------------------------
247         self.prev_cath_x = cath_info.pos_x # UPDATE PREV values before contin.
248         self.prev_cath_y = cath_info.pos_y
249         self.prev_cath_phi = cath_info.phi
250         self.prev_cath_avg_curv = cath_info.avg_curv
251
252         self.timestamp = rospy.Time.now().secs +
253             rospy.Time.now().nsecs/1000000000 # reset timestamp
254         # ------------------------------------------------------------
255         # Technically, the next line only starts one discrete step of
256         # the control-algorithm. However, this is done in a separate
257         # Thread. If it were done in the main Thread, the controller
258         # will in some occasions lag behind significantly after a while
259         # ------------------------------------------------------------
260         # DO WORK (parameters)
261         Do_Threaded_Work("Threaded-Func", cath_info, n_steps, ev_width).start()
262
263
264 class Do_Threaded_Work(threading.Thread):
265     # This implements in essence a separately threaded function "run()"
266     def __init__(self, name, cath_info, n_steps, ev_width):
267         threading.Thread.__init__(self)
268         # ------------------------------------------------------------------
269         # constructor of this class
270         # ------------------------------------------------------------------
271         self.name = name
272         self.cath_info = cath_info
273         # tells the function, how many "steps" to advance
274         self.n_steps = n_steps
275         # tells the function, how much the path has to be corrected
276         self.ev_width = ev_width
277         self.m_per_pixel = cath_info.m_per_pixel # pixel to meter conversion
278
279         self.P_const = 0.60 # P
280         self.I_const = 1.00 # not implemented
281         self.saturation_angle = 80
282         self.m_per_pixel = cath_info.m_per_pixel
283
284         self.pub_magfield = rospy.Publisher("/backward_model/field",
285             FieldStamped, queue_size=1)
```

```
286         self.pub_advancer = rospy.Publisher("/navion/target_velocity_mca",
287             Float32, queue_size=1)
288
289         # Load system calibration (for max magnetic field)
290         rp = rospkg.RosPack()
291         cal_path = os.path.join(rp.get_path('mpem'), 'cal',
292             'Navion_2_Calibration_24-02-2020.yaml')
293         self.BM = BackwardModelMPEML2()
294         self.BM.setCalibrationFile(cal_path)
295
296
297     def run(self):
298         # ---------------------------------------------------------------------
299         # The main Work is done in this Function!
300         # After all the Decision-Logic, finally this function is called. Its
301         # only purpose, is to carry out the precalculated fwd or bckwd movement
302         # and calculate the magnetic field and publish it.
303         # ---------------------------------------------------------------------
304         alpha, location = self.do_P_controller()
305         alpha = alpha /360 *2 *np.pi
306
307         # handle Field Output -------------------------------------------------
308         field_out_msg = FieldStamped()
309         field_out_msg.header.stamp = rospy.Time.now()
310         field_out_msg.header.frame_id = "mns"
311         field_out_msg.field.vector = Vector3( # this is magfield Vector [tesla]
312             0.01*np.cos(alpha),
313             0.01*np.sin(alpha),
314             0 )
315         field_out_msg.field.position = Point( # this is location of vec. in m!
316             self.m_per_pixel*location[0],
317             self.m_per_pixel*location[1],
318             0 )
319
320         # for determining the max possible field vector
321         field_temp = getMaxFieldAlignedWithTargetField(
322             self.BM,
323             np.array([self.m_per_pixel*location[0],
324                 self.m_per_pixel*location[1], 0]), # location OK
325             np.array([np.cos(alpha), np.sin(alpha), 0]),
326             np.array([35., 35., 35.]) )
327         field_out_msg.field.vector = Vector3(0.8*field_temp[0],
328             0.8*field_temp[1], 0.8*field_temp[2])
329
330         self.pub_magfield.publish(field_out_msg)
331
332         # handle Advancer Output ----------------------------------------------
333         adv_out_msg = Float32()
334         # n_steps negative means backwards, positive means forwards
335         dir = np.sign(self.n_steps)
336         # determines how many seconds to run the advancer for one step
337         timeconstant = 3.0
338         # "velocity of advancer" mm/sec times forward(+) or backwards(-)
339         adv_out_msg.data = dir*2.00
340
341         self.pub_advancer.publish(adv_out_msg)
342         # let's the advancer run for the neccessary amount of time
343         time.sleep(abs(self.n_steps)*timeconstant)
344         adv_out_msg.data = 0
345         self.pub_advancer.publish(adv_out_msg)
346
347
348     def do_P_controller(self):
349         pos_x = self.cath_info.pos_x
350         pos_y = self.cath_info.pos_y
351         phi = self.cath_info.phi
352         avg_curv = self.cath_info.avg_curv
353         path_uvec_x = self.cath_info.path_uvec_x
354         path_uvec_y = self.cath_info.path_uvec_y
355         path_orig_x = self.cath_info.path_orig_x
356         path_orig_y = self.cath_info.path_orig_y
357
```

```
358         # this vector is normal to path_uvec: defines the positive error dir.
359         posdir = [path_uvec_y, -path_uvec_x]
360         # vector from (any) point on path to catheter.
361         orig_cath_vec = [pos_x - path_orig_x, pos_y - path_orig_y]
362 
363         # yield the catheters normal distance to path
364         errdist = np.dot(posdir, orig_cath_vec) # w.r.t. Navion mns! (+/-y)
365         # width-stepsize in meters (whole channel is ~20mm = 0.02m)
366         sideways_stepsize = -0.003
367         rospy.loginfo(f"Errdistance was: {round(errdist*self.m_per_pixel, 5)}
368             and will be corrected by ev_width:
369             {round(sideways_stepsize *self.ev_width, 5)}")
370         errdist += sideways_stepsize /self.m_per_pixel *self.ev_width
371 
372         # alpha is the angle, at which the field should be
373         # angled at (w.r.t. path vector ideally).
374         alpha = -errdist * self.P_const
375 
376         if (abs(alpha) > self.saturation_angle):
377         # angles above a certain threshold don't make sense for navion...
378             alpha = np.sign(alpha) * self.saturation_angle
379 
380         # calculates, where the tip position is
381         # (w.r.t. Navion Origin and navion mns!) (in "pixels")
382         at_y = -(pos_x - self.cath_info.navion_orig_x)
383         at_x = -(pos_y - self.cath_info.navion_orig_y)
384         return alpha, [at_x, at_y]
385 
386 
387     def do_PI_controller(self, cath_info): # not implemented
388         pass
389 
390 
391 
392 # script protection ==========================================================
393 if (__name__ == "__main__"):
394     try:
395         # do work ------------------------------------------------------------
396         rospy.init_node("controller_main")
397         Controller_Main_Node()
398         rospy.spin()
399     except rospy.ROSInterruptException:
400         pass
```

A.3 X-ray Emulator-Node source-code

```
#!/usr/bin/env python
# debugging tool, should emulate the image stream, coming from the xray

import numpy as np
import numpy.ma as ma
import matplotlib.pyplot as plt
import scipy
import scipy.ndimage
import random
import time

import rospy
import sys
import cv2
from cv_bridge import CvBridge
from sensor_msgs.msg import Image



# ============================================================================
# code main body ############################################################
# ============================================================================



# convenience definitions ===================================================
bridge = CvBridge()
R = False
# filename, R = "/home/ubuntu/tesla_ws/src/epi_tracker/media/
#       Our_Fluoroscope_1.webm", 1 # mediocre footage from our xray
# filename, R = "/home/ubuntu/tesla_ws/src/epi_tracker/media/
#       Spinal_Catheter_Fluoroscope.mp4", 2 # crisp yt-video
filename, R = "/home/ubuntu/tesla_ws/src/epi_tracker/media/
        marker_fluoroscope.webm", 3 # our crisp footage



# function definition =======================================================
def start_node(filename):
    rospy.init_node("my_xray_emulator")
    rospy.loginfo("----- Xray-Emulator node has been started -----")
    pub = rospy.Publisher("emulated_xray", Image, queue_size=1)

    # retrieving the video and its metadata ----------------------------------
    vid_captured = cv2.VideoCapture(filename)
    if (vid_captured.isOpened() == False):
        print("Error opening the video file!")
    else:
        fps = vid_captured.get(5)
        frame_count = vid_captured.get(7)
        print("suceeded capturing the video file:")
        print("Frames per secon: ", fps, "fps")
        print("Frame count: ", frame_count, "total frames")

    # outermost loop! otherwise node will not exit when killed ---------------
    while not rospy.is_shutdown():

        # publish the video framewise ----------------------------------------
        if (vid_captured.isOpened()):
            # returns a tuple: bool and frame. ret = 0 for no input
            has_frame, frame = vid_captured.read()
            if (has_frame == True):

                # cropping + resize the image (match to file) ----------------
                # frameCropped = frame[:, 69:550, :] # for video 1
                # frameCropped = frame[:, 731:1880, :] # for video 2
                frameCropped = frame[5:475, 70:550, :] # for video 3

                # resize (stretch) to match xray res
```

```
70                  frameResized = cv2.resize(frameCropped, (1024, 1024))
71
72                  # emulate registration markers -------------------------------
73                  noise = random.uniform(-1, 1)
74                  err = int(noise*1024 *0.005)
75                  if (R == 1):
76                      frameReg = frameResized
77                      frameReg = cv2.circle(frameReg,
78                          (880, 150), 20, (10, 10, 10), -1)
79                      frameReg = cv2.circle(frameReg,
80                          (860, 500), 20, (10, 10, 10), -1)
81                      frameReg = cv2.circle(frameReg,
82                          (845, 700), 20, (10, 10, 10), -1)
83                      frameReg = cv2.circle(frameReg,
84                          (830, 950), 20, (10, 10, 10), -1)
85                      b = 60
86                      h = 15
87                      frameReg = cv2.rectangle(frameReg,
88                          (330, 50),  (330+h, 50+b), (10, 10, 10), -1)
89                      frameReg = cv2.rectangle(frameReg,
90                          (310, 450), (310+h, 450+b), (10, 10, 10), -1)
91                      frameReg = cv2.rectangle(frameReg,
92                          (285, 850), (285+h, 850+b), (10, 10, 10), -1)
93                  elif (R == 2):
94                      frameReg = frameResized
95                      frameReg = cv2.circle(frameReg,
96                          (410, 100), 20, (10, 10, 10), -1)
97                      frameReg = cv2.circle(frameReg,
98                          (365, 400), 20, (10, 10, 10), -1)
99                      frameReg = cv2.circle(frameReg,
100                         (335, 700), 20, (10, 10, 10), -1)
101                     frameReg = cv2.circle(frameReg,
102                         (320, 950), 20, (10, 10, 10), -1)
103                     b = 60
104                     h = 15
105                     frameReg = cv2.rectangle(frameReg,
106                         (770, 150), (770+h, 150+b), (10, 10, 10), -1)
107                     frameReg = cv2.rectangle(frameReg,
108                         (750, 500), (750+h, 500+b), (10, 10, 10), -1)
109                     frameReg = cv2.rectangle(frameReg,
110                         (740, 800), (740+h, 800+b), (10, 10, 10), -1)
111                 elif (R == 3):
112                     p = (1024/2, 1024/2)
113                     M = cv2.getRotationMatrix2D(p, 180, 1.0)
114                     frameResized = cv2.warpAffine(frameResized,
115                         M, (1024, 1024))
116
117                 else:
118                     print("no registration-marker emulation available")
119
120                 # handle publishing the edited footage ------------------------
121                 # convert img to a message: alt: rgb8, mono8, ...
122                 temp_msg = bridge.cv2_to_imgmsg(frameResized, "bgr8")
123                 temp_msg.header.frame_id = "emulated_xray"
124                 pub.publish(temp_msg)
125             else: # goes here if video ends naturally
126                 vid_captured = cv2.VideoCapture(filename) # restarts video loop
127         # ------------------------------------------------------------------
128
129         rospy.Rate(100).sleep() # roughly fixes the "playback rate" in Hz
130     vid_captured.release()
131
132
133
134 # script protection ==========================================================
135 if (__name__ == "__main__"):
136     try:
137         start_node(filename)
138     except rospy.ROSInterruptException:
139         pass
```